Precise Type Analysis on JavaScript Language Specifications via Generalized Occurrence Typing

Technical Report

ANONYMOUS AUTHOR(S)

This technical report accompanies the paper of the same title. It presents the full definitions of the base and the extended type analyses (§1–§2), the full concretization of the extended abstract domain, and the soundness theorem with its proof structure (§3). The concrete semantics, the extended analysis, and the proof are mechanized in Lean 4.

1 Base Type Analysis

This section defines the base type analysis for JavaScript language specification using an abstract interpretation [1, 2] framework with *syntactic type refinement* used in prior work [3]. It starts with the definition of $\mathsf{IR}_{\mathsf{ES}}$, the *imperative*-style untyped intermediate representation for ECMA-262, and then describes the base type analysis on it.

1.1 $\mathsf{IR}_{\mathsf{ES}}$: Untyped Intermediate Representation for ECMA-262

An $\mathsf{IR}_{\mathsf{ES}}$ program is a mechanized specification and its instructions are uniquely labeled with $l \in \mathcal{L}$.

Syntax. Figure 1 shows the syntax of $\mathsf{IR}_{\mathsf{ES}}$. A program P consists of three mappings: 1) $\mathsf{func} : \mathcal{L} \rightharpoonup \mathcal{F}$, 2) $\mathsf{inst} : \mathcal{L} \to \mathcal{I}$, and 3) $\mathsf{next} : \mathcal{L} \rightharpoonup \mathcal{L}$ that map labels to their enclosing functions, instructions, and next labels, respectively, where $\rightharpoonup$ denotes a partial function. An instruction $i \in \mathcal{I}$ is either a sequence, an assignment, a record allocation, a field update, a branch, an assertion, a function call, or a return instruction. An expression $e \in \mathcal{E}$ is either a literal, a variable, a unary/binary operation, a function, or a dynamic type check expression. We use the $\overline{X_k}$ notation to denote a sequence of elements $X_1, \ldots, X_n$ with an index k or $\overline{X}$ when the index is not important.

Types. A dynamic type check expression "$e : \tau$" checks if the expression e is an instance of the type τ. A type $\tau \in \mathcal{T}$ is either literal (e.g., `42`, `#t`, `"abc"`), a primitive (**B** for booleans, **N** for numbers, **S** for strings), a function, a record, a union, an intersection, a difference, or a top/bottom type. We define a type concretization γ_τ and following auxiliary operations.

- **Type Concretization**: $\gamma_\tau : \mathcal{T} \to \mathcal{P}(\mathbb{H} \times \mathbb{V})$

$$\begin{array}{lll} \gamma_\tau(p) = \mathbb{H} \times \{p\} & \gamma_\tau(\Lambda) = \mathbb{H} \times \{\Lambda\} & \gamma_\tau(\tau_1 \setminus \tau_2) = \gamma_\tau(\tau_1) \setminus \gamma_\tau(\tau_2) \\ \gamma_\tau(\mathbf{B}) = \mathbb{H} \times \mathbb{B} & \gamma_\tau(\tau_1 \sqcap \tau_2) = \gamma_\tau(\tau_1) \cap \gamma_\tau(\tau_2) & \gamma_\tau(\top) = \mathbb{H} \times \mathbb{V} \\ \gamma_\tau(\mathbf{N}) = \mathbb{H} \times \mathbb{N} & \gamma_\tau(\tau_1 \sqcup \tau_2) = \gamma_\tau(\tau_1) \cup \gamma_\tau(\tau_2) & \gamma_\tau(\bot) = \varnothing \end{array}$$

$$\gamma_\tau(\{\overline{\mathsf{f}_k : \tau_k}\}) = \{(h, a) \mid a \in \mathbb{A} \wedge h \in \mathbb{H} \wedge h(a) = \{\overline{\mathsf{f}'_i : v_i}\} \wedge \forall k. \exists i. (\mathsf{f}_k = \mathsf{f}'_i \wedge (h, v_i) \in \gamma_\tau(\tau_k))\}$$

- **Subtype Relation**: $<: \subseteq \mathcal{T} \times \mathcal{T}$

$$\tau_1 <: \tau_2 \iff \gamma_\tau(\tau_1) \subseteq \gamma_\tau(\tau_2)$$

- **Field Access**: $\tau.\mathsf{f} \in \mathcal{T}$

$$\begin{array}{ll} p.\mathsf{f} = \top & \\ \mathbf{B}.\mathsf{f} = \top & \{\overline{\mathsf{f}_k : \tau_k}\}.\mathsf{f} = \begin{cases} \tau_k & \text{if } \mathsf{f} = \mathsf{f}_k \\ \top & \text{otherwise} \end{cases} \\ \mathbf{N}.\mathsf{f} = \top & \end{array} \qquad \begin{array}{ll} (\tau_1 \sqcup \tau_2).\mathsf{f} = \tau_1.\mathsf{f} \sqcup \tau_2.\mathsf{f} & \Lambda.\mathsf{f} = \top \\ (\tau_1 \sqcap \tau_2).\mathsf{f} = \tau_1.\mathsf{f} \sqcap \tau_2.\mathsf{f} & \top.\mathsf{f} = \top \\ (\tau_1 \setminus \tau_2).\mathsf{f} = \tau_1.\mathsf{f} & \bot.\mathsf{f} = \bot \end{array}$$

Programs	$P = (\text{func}, \text{inst}, \text{next})$	**Functions**	$\mathcal{F} \ni \Lambda ::= \lambda(\overline{x}).l$
Instructions	$\mathcal{I} \ni i ::= x = e \mid x = \{\overline{\mathtt{f} : e}\} \mid e.\mathtt{f} = e$	**Labels**	$\mathcal{L} \ni l$
	$\mid \mathtt{if}\ (e)\ l\ \mathtt{else}\ l \mid \mathtt{assert}\ e$	**Variables**	$\mathcal{X} \ni x$
	$\mid x = e(\overline{e}) \mid \mathtt{return}\ e$	**Fields**	$\mathbb{F} \ni \mathtt{f}$
Expressions	$\mathcal{E} \ni e ::= p \mid x \mid e.\mathtt{f} \mid \ominus e \mid e \oplus e \mid \Lambda \mid e : \tau$	**Primitives**	$\mathbb{P} \ni p ::= b \mid n \mid s$
Types	$\mathcal{T} \ni \tau ::= p \mid \mathbf{B} \mid \mathbf{N} \mid \mathbf{S} \mid \Lambda \mid \{\overline{\mathtt{f} : \tau}\}$	**Booleans**	$\mathbb{B} \ni b ::= \#\mathtt{t} \mid \#\mathtt{f}$
	$\mid \tau \sqcup \tau \mid \tau \sqcap \tau \mid \tau \backslash \tau \mid \top \mid \bot$	**Numbers**	$\mathbb{N} \ni n$
		Strings	$\mathbb{S} \ni s$

Fig. 1. Syntax of the *imperative*-style untyped intermediate representation IR_{ES} for ECMA-262.

States	$\sigma \in \Sigma = \mathbb{K} \times \mathcal{L} \times \mathbb{M}$	**Calling Contexts**	$\kappa \in \mathbb{K} = \{\epsilon\} \uplus \Sigma$
Memories	$m \in \mathbb{M} = \mathbb{H} \times \mathbb{E}$	**Heaps**	$h \in \mathbb{H} = \mathbb{A} \rightharpoonup \mathbb{R}$
Environments	$\rho \in \mathbb{E} = \mathcal{X} \rightharpoonup \mathbb{V}$	**Addresses**	$a \in \mathbb{A}$
Values	$v \in \mathbb{V} = \mathbb{P} \uplus \mathbb{A} \uplus \mathcal{F}$	**Records**	$r \in \mathbb{R} = \mathbb{F} \rightharpoonup \mathbb{V}$

Fig. 2. States of IR_{ES} programs.

- **Field Update**: $\tau[\mathtt{f} : \tau] \in \mathcal{T}$

$$
\begin{array}{lll}
p[\mathtt{f} : \tau] = \bot & (\tau_1 \sqcup \tau_2)[\mathtt{f} : \tau_f] = \tau_1[\mathtt{f} : \tau_f] \sqcup \tau_2[\mathtt{f} : \tau_f] & \Lambda[\mathtt{f} : \tau] = \bot \\
\mathbf{B}[\mathtt{f} : \tau] = \bot & (\tau_1 \sqcap \tau_2)[\mathtt{f} : \tau_f] = \tau_1[\mathtt{f} : \tau_f] \sqcap \tau_2[\mathtt{f} : \tau_f] & \top[\mathtt{f} : \tau] = \{\mathtt{f} : \tau\} \\
\mathbf{N}[\mathtt{f} : \tau] = \bot & (\tau_1 \backslash \tau_2)[\mathtt{f} : \tau_f] = \tau_1[\mathtt{f} : \tau_f] \backslash \tau_2[\mathtt{f} : \tau_f] & \bot[\mathtt{f} : \tau] = \bot
\end{array}
$$

States. As described in Figure 2, a program state $\sigma \in \Sigma$ is a tuple of a calling context, a label, and a memory. A calling context $\kappa \in \mathbb{K}$ is an empty context ϵ or a state at the call site. A memory $m \in \mathbb{M}$ is a tuple of a heap and an environment. A heap $h \in \mathbb{H}$ is a finite map from addresses to records. An address $a \in \mathbb{A}$ is a unique identifier for each record. A record $r \in \mathbb{R}$ is a finite map from fields to values. An environment $\rho \in \mathbb{E}$ is a finite map ($\rightharpoonup$) from variables to values. A value $v \in \mathbb{V}$ is either a primitive, an address, or a function value. Note that variables and fields are *mutable*, and function values do not capture the current environment.

Semantics. The concrete semantics of the core language is defined as a state transition system $(\rightsquigarrow, \Sigma, \Sigma_\iota)$ where Σ_ι is a set of initial states and $\rightsquigarrow \subseteq \Sigma \times \Sigma$ is a transition relation between states:

$$\sigma \rightsquigarrow \sigma' \iff [\![i]\!]_i(\sigma) = \sigma' \quad \text{where} \quad \sigma = (l, _, _, _, _) \wedge i = \text{inst}(l)$$

The *collecting semantics* is defined as a set of reachable states from the initial states and can be computed by a fixed-point computation of a transfer function $F : \mathcal{P}(\Sigma) \to \mathcal{P}(\Sigma)$:

$$[\![P]\!] = \{\sigma \in \Sigma \mid \exists \sigma_\iota \in \Sigma_\iota.\ \sigma_\iota \rightsquigarrow^* \sigma\} = \mathbf{lfp} F = \lim_{n \to \infty} F^n(\varnothing) \quad \text{where} \quad F(S) = \Sigma_\iota \cup \bigcup_{\sigma \in S} \{\sigma' \mid \sigma \rightsquigarrow \sigma'\}$$

$$\boxed{[\![-]\!]_i : \Sigma \to \Sigma}$$

$$
\begin{array}{lll}
[\![x = e]\!]_i(\sigma) & = (\kappa, \mathsf{next}(l), m_1) & \text{where } m_1 = m.\mathsf{update}_x(x, [\![e]\!]_e(m)) \\
[\![x = \{\overline{\mathtt{f}_k : e_k}\}]\!]_i(\sigma) & = (\kappa, \mathsf{next}(l), m_2) & \text{where } \begin{cases} (a, m_1) = m.\mathsf{alloc}(\{\overline{\mathtt{f}_k : [\![e_k]\!]_e(m)}\}) \\ m_2 \quad\;\; = m_1.\mathsf{update}_x(x, a) \end{cases} \\
[\![e_1.\mathtt{f} = e_2]\!]_i(\sigma) & = (\kappa, \mathsf{next}(l), m_1) & \text{where } m_1 = m.\mathsf{update}_{\mathtt{f}}([\![e_1]\!]_e(m), \mathtt{f}, [\![e_2]\!]_e(m)) \\
[\![\mathtt{if}\ (e)\ l_1\ \mathtt{else}\ l_2]\!]_i(\sigma) & = (\kappa, l', m) & \text{where } l' = \begin{cases} l_1 \text{ if } [\![e]\!]_e(m) = \#\mathtt{t} \\ l_2 \text{ if } [\![e]\!]_e(m) = \#\mathtt{f} \end{cases} \\
[\![\mathtt{assert}(e)]\!]_i(\sigma) & = (\kappa, \mathsf{next}(l), m) & \text{if} \quad [\![e]\!]_e(m) = \#\mathtt{t} \\
[\![\mathtt{call}\ x = e_0(\overline{e_k})]\!]_i(\sigma) & = (\kappa_0, l_0, m_1) & \text{where } \begin{cases} \kappa_0 \quad\;\; = \sigma \\ [\![e_0]\!]_e(m) = \lambda(\overline{x_k}).l_0 \\ m_1 \quad\;\; = (h, \{\overline{x_k \mapsto [\![e_k]\!]_e(m)}\}) \end{cases} \\
[\![\mathtt{return}\ e]\!]_i(\sigma) & = (\kappa_1, \mathsf{next}(l_1), m_3,) & \text{where } \begin{cases} \kappa \quad\;\; = (\kappa_1, l_1, (_, \rho_1)) \\ \mathsf{inst}(l_1) = \mathtt{call}\ x = _; \\ m_2 \quad\;\; = (h, \rho_1) \\ m_3 \quad\;\; = m_2.\mathsf{update}_x(x, [\![e]\!]_e(m)) \end{cases}
\end{array}
$$

$$\boxed{[\![-]\!]_e : \mathbb{M} \to \mathbb{V}}$$

$$
\begin{array}{llll}
[\![p]\!]_e(m) = p & [\![e.\mathtt{f}]\!]_e(m) & = h([\![e]\!]_e(m))(\mathtt{f}) & \\
[\![x]\!]_e(m) = \rho(x) & [\![\ominus\ e]\!]_e(m) & = \ominus\ [\![e]\!]_e(m) & [\![e : \tau]\!]_e(m) = \begin{cases} \#\mathtt{t} \text{ if } (h, [\![e]\!]_e(m)) \in \gamma_\tau(\tau) \\ \#\mathtt{f} \text{ otherwise} \end{cases} \\
[\![\Lambda]\!]_e(m) = \Lambda & [\![e_1 \oplus e_2]\!]_e(m) & = [\![e_1]\!]_e(m) \oplus [\![e_2]\!]_e(m) &
\end{array}
$$

Fig. 3. Semantics of instructions and expressions of $\mathsf{IR}_{\mathsf{ES}}$ where $\sigma = (\kappa, l, m)$ and $m = (h, \rho)$.

The transition-style semantics of instructions $[\![-]\!]_i : \Sigma \to \Sigma$ and expressions $[\![-]\!]_e : \mathbb{M} \to \mathbb{V}$ are defined in Figure 3 and we use the following auxiliary operations:

$$
\begin{array}{llll}
\sigma.\mathsf{caller} = \kappa & \kappa.\mathsf{state} = \sigma & m.\mathsf{heap} & = h \\
\sigma.\mathsf{label} = l & \kappa.\mathsf{caller} = \sigma.\mathsf{caller} & m.\mathsf{env} & = \rho \\
\sigma.\mathsf{mem} = m & \kappa.\mathsf{label} = \sigma.\mathsf{label} & m.\mathsf{update}_x(x, v) & = (h, \rho[x \mapsto v]) \\
\sigma.\mathsf{heap} = m.\mathsf{heap} & \kappa.\mathsf{mem} = \sigma.\mathsf{mem} & m.\mathsf{update}_{\mathtt{f}}(a, \mathtt{f}, v) & = (h[a \mapsto h(a)[\mathtt{f} \mapsto v]], \rho) \\
\sigma.\mathsf{env} = m.\mathsf{env} & \kappa.\mathsf{heap} = \sigma.\mathsf{heap} & m.\mathsf{alloc}(r) & = (a, (h[a \mapsto r], \rho)) \\
\text{where } \sigma = (\kappa, l, m) & \kappa.\mathsf{env} = \sigma.\mathsf{env} & & \text{where } a \text{ is a fresh address in } h \\
 & \kappa.\mathsf{var} = x & \text{where } m = (h, \rho) & \\
 & \text{where } \kappa = (\sigma, x) & &
\end{array}
$$

1.2 Type Analysis for $\mathsf{IR}_{\mathsf{ES}}$

We first define flow-sensitive abstract states and then define abstract semantics for the type analysis on a given $\mathsf{IR}_{\mathsf{ES}}$ program.

Abstract Domains. Figure 4 shows the abstract domains for each component of the type analysis. An abstract state $\widehat{\sigma} \in \widehat{\Sigma}$ is a pair of an abstract calling context and a flow-sensitive typing result. An abstract calling context $\widehat{\kappa} \in \widehat{\mathbb{K}}$ is a mapping from each function to a set of call-site labels. A flow-sensitive typing result $\xi \in \Xi$ is a mapping from labels to typing results. A typing result $\Gamma \in \mathbb{\Gamma}$ is a pair of an abstract environment and an effect. An abstract environment $\widehat{\rho} \in \widehat{\mathbb{E}}$ is a mapping

Abstract States	$\widehat{\sigma} \in \widehat{\Sigma}$	$=$	$\widehat{\mathbb{K}} \times \Xi$
Abstract Calling Contexts	$\widehat{\kappa} \in \widehat{\mathbb{K}}$	$=$	$\mathcal{F} \to \mathcal{P}(\mathcal{L})$
Flow-Sensitive Typing Results	$\xi \in \Xi$	$=$	$\mathcal{L} \to \mathbb{\Gamma}$
Typing Results	$\Gamma \in \mathbb{\Gamma}$	$=$	$\widehat{\mathbb{E}} \times \Delta$
Abstract Environments	$\widehat{\rho} \in \widehat{\mathbb{E}}$	$=$	$\mathcal{X} \to \widehat{\mathbb{V}}$
Abstract Values	$\widehat{v} \in \widehat{\mathbb{V}}$	$=$	$\mathcal{T}$
Effects	$\delta \in \Delta$	$=$	$\mathbb{F} \to \mathcal{T}$

Fig. 4. Abstract domains for each component of the type analysis.

$$\gamma_\sigma : \widehat{\Sigma} \to \mathcal{P}(\Sigma) \quad \gamma_\kappa : \widehat{\mathbb{K}} \to \mathcal{P}(\Sigma) \quad \gamma_\xi : \Xi \to \mathcal{P}(\Sigma) \quad \gamma_\Gamma : \mathbb{\Gamma} \to \mathcal{P}(\Sigma)$$
$$\gamma_\rho : \widehat{\mathbb{E}} \to \mathcal{P}(\mathbb{M}) \quad \gamma_v : \widehat{\mathbb{V}} \to \mathcal{P}(\mathbb{H} \times \mathbb{V}) \quad \gamma_\delta : \Delta \to \mathcal{P}(\Sigma)$$

$$\begin{aligned}
\gamma_\sigma(\widehat{\sigma}) &= \{\sigma \mid \widehat{\sigma} = (\widehat{\kappa}, \xi) \wedge \sigma \in \gamma_\kappa(\widehat{\kappa}) \wedge \sigma \in \gamma_\xi(\xi)\} \\
\gamma_\kappa(\widehat{\kappa}) &= \{\sigma \mid \kappa = \sigma.\mathsf{caller} \wedge (\kappa = \epsilon \vee (\kappa.\mathsf{state} \in \gamma_\kappa(\widehat{\kappa}) \wedge (\kappa.\mathsf{label}, \kappa.\mathsf{var}) \in \widehat{\kappa} \circ \mathsf{func}(l)))\} \\
\gamma_\xi(\xi) &= \{\sigma \mid \sigma \in \gamma_\Gamma \circ \xi(\sigma.\mathsf{label}) \wedge \kappa = \sigma.\mathsf{caller} \wedge (\kappa = \epsilon \vee \kappa.\mathsf{state} \in \gamma_\xi(\xi))\} \\
\gamma_\Gamma(\Gamma) &= \{\sigma \mid \Gamma = (\widehat{\rho}, \delta) \wedge \sigma.\mathsf{mem} \in \gamma_\rho(\widehat{\rho}) \wedge \sigma \in \gamma_\delta(\delta)\} \\
\gamma_\rho(\widehat{\rho}) &= \{m \mid m = (h, \rho) \wedge \forall (x \mapsto v) \in \rho.\ (h, v) \in \gamma_v \circ \widehat{\rho}(x)\} \\
\gamma_v(\tau) &= \gamma_\tau(\tau) \\
\gamma_\delta(\delta) &= \{\sigma \mid \kappa = \sigma.\mathsf{caller} \wedge (\kappa = \epsilon \vee (\kappa.\mathsf{heap}, \sigma.\mathsf{heap}) \in_\delta \delta)\}
\end{aligned}$$

where $(h, h') \in_\delta \delta \iff \forall a \mapsto r \in h.\ \forall \mathsf{f} \mapsto v \in r.\ v' = h'(a)(\mathsf{f}) \wedge (v' \neq v \Rightarrow (h', v') \in \gamma_\tau \circ \delta(\mathsf{f}))$

$$\boxed{\widehat{d} \sqsubseteq \widehat{d}}$$

$$\widehat{d}_1 \sqsubseteq \widehat{d}_2 \iff \gamma_d(\widehat{d}_1) \subseteq \gamma_d(\widehat{d}_2)$$

$$\boxed{\bot_d}$$

$$\begin{aligned}
&\bot_\sigma = (\bot_\kappa, \bot_\xi) && \bot_\xi = \{l \mapsto \bot_\Gamma \mid l \in \mathcal{L}\} && \bot_\rho = \{x \mapsto \bot_v \mid x \in \mathcal{X}\} && \bot_v = \bot \\
&\bot_\kappa = \{\Lambda \mapsto \varnothing \mid \Lambda \in \mathcal{F}\} && \bot_\Gamma = (\bot_\rho, \bot_\delta) && \bot_\delta = \{\mathsf{f} \mapsto \bot \mid \mathsf{f} \in \mathbb{F}\}
\end{aligned}$$

$$\boxed{\widehat{d} \sqcup \widehat{d} = \widehat{d}}$$

$$\Gamma_1 \sqcup \Gamma_2 = (\widehat{\rho}_1 \sqcup \widehat{\rho}_2, \delta_1 \sqcup \delta_2) \text{ where } \Gamma_1 = (\widehat{\rho}_1, \delta_1) \text{ and } \Gamma_2 = (\widehat{\rho}_2, \delta_2)$$

$$\begin{aligned}
\widehat{\sigma}_1 \sqcup \widehat{\sigma}_2 &= (\widehat{\kappa}_1 \sqcup \widehat{\kappa}_2, \xi_1 \sqcup \xi_2) & \widehat{\rho}_1 \sqcup \widehat{\rho}_2 &= \{x \mapsto \widehat{\rho}_1(x) \sqcup \widehat{\rho}_2(x) \mid x \in \mathcal{X}\} \\
\widehat{\kappa}_1 \sqcup \widehat{\kappa}_2 &= \{\Lambda \mapsto \widehat{\kappa}_1(\Lambda) \cup \widehat{\kappa}_2(\Lambda) \mid \Lambda \in \mathcal{F}\} & \delta_1 \sqcup \delta_2 &= \{\mathsf{f} \mapsto \delta_1(\mathsf{f}) \sqcup \delta_2(\mathsf{f}) \mid \mathsf{f} \in \mathbb{F}\} \\
\xi_1 \sqcup \xi_2 &= \{l \mapsto \xi_1(l) \sqcup \xi_2(l) \mid l \in \mathcal{L}\} & \widehat{v}_1 \sqcup \widehat{v}_2 &= \widehat{v}_1 \sqcup \widehat{v}_2
\end{aligned}$$

$$\boxed{\widehat{d} \sqcap \widehat{d} = \widehat{d}}$$

$$\Gamma_1 \sqcap \Gamma_2 = (\widehat{\rho}_1 \sqcap \widehat{\rho}_2, \delta_1 \sqcap \delta_2) \text{ where } \Gamma_1 = (\widehat{\rho}_1, \delta_1) \text{ and } \Gamma_2 = (\widehat{\rho}_2, \delta_2)$$

$$\begin{aligned}
\widehat{\sigma}_1 \sqcap \widehat{\sigma}_2 &= (\widehat{\kappa}_1 \sqcap \widehat{\kappa}_2, \xi_1 \sqcap \xi_2) & \widehat{\rho}_1 \sqcap \widehat{\rho}_2 &= \{x \mapsto \widehat{\rho}_1(x) \sqcap \widehat{\rho}_2(x) \mid x \in \mathcal{X}\} \\
\widehat{\kappa}_1 \sqcap \widehat{\kappa}_2 &= \{\Lambda \mapsto \widehat{\kappa}_1(\Lambda) \cup \widehat{\kappa}_2(\Lambda) \mid \Lambda \in \mathcal{F}\} & \delta_1 \sqcap \delta_2 &= \{\mathsf{f} \mapsto \delta_1(\mathsf{f}) \sqcap \delta_2(\mathsf{f}) \mid \mathsf{f} \in \mathbb{F}\} \\
\xi_1 \sqcap \xi_2 &= \{l \mapsto \xi_1(l) \sqcap \xi_2(l) \mid l \in \mathcal{L}\} & \widehat{v}_1 \sqcap \widehat{v}_2 &= \widehat{v}_1 \sqcap \widehat{v}_2
\end{aligned}$$

Fig. 5. Concretization (γ), partial order ($\sqsubseteq$), bottom ($\bot$), join ($\sqcup$), and meet ($\sqcap$) operations on abstract domains.

$$\widehat{[\![-]\!]}_i : (\mathcal{L} \times \widehat{\Sigma}) \to \widehat{\Sigma}$$

$$\begin{array}{lll}
\widehat{[\![x = e]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathsf{update}}_x(x, \widehat{[\![e]\!]}_e(\Gamma)) \\
\widehat{[\![x = \{\mathtt{f}_k : e_k\}]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathsf{update}}_x(x, \{\mathtt{f}_k : \widehat{[\![e_k]\!]}_e(\Gamma)\}) \\
\widehat{[\![e_1.\mathtt{f} = e_2]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathsf{update}}_{\mathtt{f}}(\widehat{[\![e_1]\!]}_e(\Gamma), \mathtt{f}, \widehat{[\![e_2]\!]}_e(\Gamma)) \\
\widehat{[\![\mathtt{if}\ (e)\ l_1\ \mathtt{else}\ l_2]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{l_t \mapsto \Gamma_t, l_f \mapsto \Gamma_f\}) & \text{where } \begin{cases} \Gamma_t = \Gamma.\widehat{\mathsf{refine}}(e, \#\mathtt{t}) \\ \Gamma_f = \Gamma.\widehat{\mathsf{refine}}(e, \#\mathtt{f}) \end{cases} \\
\widehat{[\![\mathtt{assert}(e)]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathsf{refine}}(e, \#\mathtt{t}) \\
\widehat{[\![\mathtt{call}\ x = e_0(\overline{e_k})]\!]}_i(l,\widehat{\sigma}) & = (\widehat{\kappa}_1, \{\overline{l_j \mapsto \Gamma_j}\}) & \text{where } \begin{cases} \widehat{v}_0 = \widehat{[\![e_0]\!]}_e(\Gamma) \\ \{\overline{\Lambda_j}\} = \{\overline{\lambda(\overline{x_{j,k}}).l_j}\} = \gamma_v(\widehat{v}_0) \cap \mathcal{F} \\ \Gamma_j = (\{\overline{x_{j,k} \mapsto \widehat{[\![e_k]\!]}_e(\Gamma)}\}, \bot_\delta) \\ \widehat{\kappa}_1 = \{\overline{\Lambda_j \mapsto \{l\}}\} \end{cases} \\
\widehat{[\![\mathtt{return}\ e]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{\overline{\mathsf{next}(l_k) \mapsto \Gamma'_k}\}) & \text{where } \begin{cases} \{\overline{l_k}\} = \widehat{\kappa} \circ \mathsf{func}(l) \\ \mathsf{inst}(l_k) = \mathtt{call}\ x_k = _; \\ \Gamma_k = \xi(l_k).\widehat{\mathsf{apply}}(\delta) \\ \Gamma'_k = \Gamma_k.\widehat{\mathsf{update}}_x(x_k, \widehat{[\![e]\!]}_e(\Gamma)) \end{cases}
\end{array}$$

$$\widehat{[\![-]\!]}_e : \mathbb{\Gamma} \to \widehat{\mathbb{V}}$$

$$\widehat{[\![p]\!]}_e(\Gamma) = p \quad \widehat{[\![\Lambda]\!]}_e(\Gamma) = \Lambda \quad \widehat{[\![x]\!]}_e(\Gamma) = \widehat{\rho}(x) \quad \widehat{[\![e.\mathtt{f}]\!]}_e(\Gamma) = \widehat{[\![e]\!]}_e(\Gamma).\mathtt{f} \quad \widehat{[\![\ominus e]\!]}_e(\Gamma) = \ominus\widehat{[\![e]\!]}_e(\Gamma) \quad \widehat{[\![e_1 \oplus e_2]\!]}_e(\Gamma) = \widehat{[\![e_1]\!]}_e(\Gamma) \oplus \widehat{[\![e_2]\!]}_e(\Gamma) \quad \widehat{[\![e : \tau]\!]}_e(\Gamma) = \mathbf{B}$$

Fig. 6. Abstract semantics of instructions and expressions of $\mathsf{IR}_{\mathsf{ES}}$ where $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ and $\xi(l) = \Gamma = (\widehat{\rho}, \delta)$.

from variables to abstract values. An abstract value $\widehat{v} \in \widehat{\mathbb{V}}$ is a type. Finally, an effect $\delta \in \Delta$ is a mapping from fields to types that represents which fields may be updated with new types in the current calling context. Effects are used to propagate the may-update information of fields into the calling context when returning from a function call for soundness. The partial order $\sqsubseteq$, join $\sqcup$, meet $\sqcap$ operations, and the remove $\setminus$ operation are defined in a point-wise manner for each component of the abstract domain. We define operations (concretization (γ), partial order ($\sqsubseteq$), bottom ($\bot$), join ($\sqcup$), and meet ($\sqcap$)) on abstract domains in a point-wise manner. Their definitions are defined in Figure 5.

Abstract Semantics. The abstract semantics $\widehat{[\![P]\!]}$ of a program is defined as a fixed-point computation of the abstract transfer function $\widehat{F} : \widehat{\Sigma} \to \widehat{\Sigma}$ from an initial abstract state $\widehat{\sigma}_\iota$:

$$\widehat{[\![P]\!]} = \mathbf{lfp}\widehat{F} = \lim_{n\to\infty} (\widehat{F})^n(\bot_\sigma) \qquad \text{where} \qquad \widehat{F}(\widehat{\sigma}) = \widehat{\sigma}_\iota \sqcup \left(\bigsqcup_{l \in \mathcal{L}} \widehat{[\![\mathsf{inst}(l)]\!]}_i(l, \widehat{\sigma}) \right)$$

where $\widehat{\sigma}_\iota$ satisfies the constraints $\Sigma_\iota \subseteq \gamma_\sigma(\widehat{\sigma}_\iota)$ and $\widehat{[\![-]\!]}_i$ and $\widehat{[\![-]\!]}_e$ are the abstract semantics of instructions and expressions, respectively, defined in Figure 6.

For abstract semantics, we introduce the auxiliary operations for typing results $\Gamma = (\widehat{\rho}, \delta) \in \mathbb{\Gamma}$:

- $\Gamma.\widehat{\mathsf{update}}_x : (\mathcal{X} \times \widehat{\mathbb{V}}) \to \mathbb{\Gamma}$ updates a variable in a typing result with a new abstract value.

$$\Gamma.\widehat{\mathsf{update}}_x(x, \widehat{v}) = (\widehat{\rho}[x \mapsto \widehat{v}], \delta)$$

- $\Gamma.\widehat{\mathsf{update}_\mathsf{f}} : (\widehat{\mathbb{V}} \times \mathbb{F} \times \widehat{\mathbb{V}}) \to \mathbb{\Gamma}$ updates a field of an abstract value in a typing result.

$$\Gamma.\widehat{\mathsf{update}_\mathsf{f}}(_, \mathsf{f}, \widehat{v}_2) = (\{x \mapsto (\widehat{v} \sqcup \widehat{v}[\mathsf{f} : \widehat{v}_2]) \mid (x \mapsto \widehat{v}) \in \widehat{\rho}\}, \delta[\mathsf{f} : \widehat{v}_2])$$

- $\Gamma.\widehat{\mathsf{apply}} : \Delta \to \mathbb{\Gamma}$ applies an effect (may-update information of fields) to a typing result.

$$\Gamma.\widehat{\mathsf{apply}}(\delta) = \bigsqcup\{\Gamma.\widehat{\mathsf{update}_\mathsf{f}}(\top_{\widehat{v}}, \mathsf{f}, \tau) \mid (\mathsf{f} : \tau) \in \delta\}$$

- $\Gamma.\widehat{\mathsf{refine}} : (\mathcal{E} \times \mathbb{B}) \to \mathbb{\Gamma}$ performs a *syntactic type refinement* on a typing result in branch/assertion instructions with conditional expressions as highlighted in Figure 6.

$$\Gamma.\widehat{\mathsf{refine}}(e_1 \;\&\&\; e_2, b) = \begin{cases} \Gamma_1 \sqcap \Gamma_2 \text{ if } b \\ \Gamma_1 \sqcup \Gamma_2 \text{ if } \neg b \end{cases} \qquad \Gamma.\widehat{\mathsf{refine}}(!\,e, b) = \Gamma.\widehat{\mathsf{refine}}(e, \neg b)$$

$$\Gamma.\widehat{\mathsf{refine}}(e_1 \;||\; e_2, b) = \begin{cases} \Gamma_1 \sqcup \Gamma_2 \text{ if } b \\ \Gamma_1 \sqcap \Gamma_2 \text{ if } \neg b \end{cases} \qquad \Gamma.\widehat{\mathsf{refine}}(x : \tau, b) = \begin{cases} (\widehat{\rho}[x \mapsto \widehat{\rho}(x) \sqcap \tau], \delta) \text{ if } b \\ (\widehat{\rho}[x \mapsto \widehat{\rho}(x) \setminus \tau], \delta) \quad \text{if } \neg b \end{cases}$$

  where $\Gamma_1 = \Gamma.\widehat{\mathsf{refine}}(e_1, b)$ and $\Gamma_2 = \Gamma.\widehat{\mathsf{refine}}(e_2, b)$.

We prove the soundness of the type analysis by showing that the abstract semantics is a sound approximation of the concrete semantics.

Theorem 1.1 (Soundness of $\widehat{[\![-]\!]}$). *For a given program P and an initial abstract state $\widehat{\sigma}_\iota$ satisfying $\Sigma_\iota \subseteq \gamma_\sigma(\widehat{\sigma}_\iota)$, the abstract semantics $\widehat{[\![P]\!]}$ is a sound approximation of the concrete semantics $[\![P]\!]$:*

$$[\![P]\!] \subseteq \gamma_\sigma(\widehat{[\![P]\!]})$$

Proof. First, we will prove that the following statement holds by induction on $k \geq 0$:

$$F^k(\varnothing) \subseteq \gamma_\sigma \circ \widehat{F}^k(\bot_\sigma) \tag{1}$$

If $k = 0$, the statement holds by definition of γ_σ and $\bot_\sigma$. Then, we assume that the statement holds for k and prove that it holds for $k + 1$. By the definition of F and $\widehat{F}$, we have:

$$\begin{aligned} F^{k+1}(\varnothing) &= F \circ F^k(\varnothing) \\ &\subseteq F \circ \gamma_\sigma \circ \widehat{F}^k(\bot_\sigma) \quad (\because \text{ I.H. and the monotonicity of } F) \\ &\subseteq \gamma_\sigma \circ \widehat{F} \circ \widehat{F}^k(\bot_\sigma) \quad (\because \text{ Lemma 1.2}) \\ &= \gamma_\sigma \circ \widehat{F}^{k+1}(\bot_\sigma) \end{aligned}$$

Therefore, according to Eq. (1), we have:

$$[\![P]\!] = \lim_{k\to\infty} F^k(\varnothing) \subseteq \lim_{k\to\infty} \gamma_\sigma \circ \widehat{F}^k(\bot_\sigma) = \gamma_\sigma \circ \widehat{[\![P]\!]} \tag{2}$$

□

Lemma 1.2 (Soundness of $\widehat{F}$). *For any abstract state $\widehat{\sigma} \in \widehat{\Sigma}$,*

$$F \circ \gamma_\sigma(\widehat{\sigma}) \subseteq \gamma_\sigma \circ \widehat{F}(\widehat{\sigma})$$

Proof. TODO

Lemma 1.3 (Soundness of $\widehat{[\![-]\!]}_i$). *For any instruction $i \in \mathcal{I}$ and abstract state $\widehat{\sigma} \in \widehat{\Sigma}$,*

$$\forall \sigma = (l, _, _, _, _) \in \gamma_\sigma(\widehat{\sigma}).\ [\![inst(l)]\!]_i(\sigma) \in \gamma_\sigma \circ \widehat{[\![i]\!]}_i(l, \widehat{\sigma})$$

Proof. TODO

Lemma 1.4 (Soundness of $\widehat{[\![-]\!]}_e$). *For any expression $e \in \mathcal{E}$ and abstract environment $\widehat{\rho} \in \widehat{\mathbb{E}}$,*

$$\forall m = (h, _) \in \gamma_\rho(\widehat{\rho}).\ (h, [\![e]\!]_e(m)) \in \gamma_v \circ \widehat{[\![e]\!]}_e(\widehat{\rho})$$

Extended Abstract State	$\widehat{\sigma} \in \widehat{\Sigma}$	$=$	$\widehat{\mathbb{K}} \times \Xi$
Abstract Calling Contexts	$\widehat{\kappa} \in \widehat{\mathbb{K}}$	$=$	$\mathcal{F} \to \mathcal{P}(\mathcal{L})$
Flow-Sensitive Typing Results	$\xi \in \Xi$	$=$	$\mathcal{L} \to \mathbb{\Gamma}$
Typing Results	$\Gamma \in \mathbb{\Gamma}$	$=$	$\widehat{\mathbb{E}} \times \Psi \times \Delta$
Abstract Environments	$\widehat{\rho} \in \widehat{\mathbb{E}}$	$=$	$\mathcal{X} \to \widehat{\mathbb{V}}$
Abstract Values	$\widehat{v} \in \widehat{\mathbb{V}}$	$=$	$\mathcal{T}^{\omega} \times \mathbb{G}$
Effects	$\delta \in \Delta$	$=$	$\mathbb{F} \to \mathcal{T}$
Type Propositions	$\psi \in \Psi$	$::=$	$\top_{\psi} \mid \bot_{\psi} \mid x : \tau \mid \omega <: \tau \mid \psi \wedge \psi \mid \ldots$
Symbols (Type Variables)	$\omega \in \Omega$	$=$	$\{\omega^{0}, \omega^{1}, \ldots\}$
Symbolic Types	$\tau^{\omega} \in \mathcal{T}^{\omega}$	$::=$	$\tau \mid \omega \mid \tau^{\omega}.\mathtt{f} \mid \{\overline{\mathtt{f} : \tau^{\omega}}\}$
Type Guards	$G \in \mathbb{G}$	$=$	$\mathcal{T}^{T} \to \Psi$
Target Types	$\tau^{T} \in \mathcal{T}^{T}$	$\subset$	$\mathcal{T}$

Fig. 7. Abstract domains for extended type analysis with occurrence typing.

Proof. The proof is by induction on the structure of expressions with $m = (h, \rho) \in \gamma_{\rho}(\widehat{\rho})$:

Lemma 1.5 (Join Operation). *For any abstract domain $\widehat{\mathbb{D}}$ and and concretization function γ,*

$$\forall \widehat{d_1}, \widehat{d_2} \in \widehat{\mathbb{D}}.\ \gamma(\widehat{d_1}) \cup \gamma(\widehat{d_2}) \subseteq \gamma(\widehat{d_1} \sqcup \widehat{d_2})$$

Proof. TODO

2 Extended Type Analysis with Generalized Occurrence Typing

This section extends type analysis with our generalized occurrence typing for the intermediate representation $\mathsf{IR}_{\mathsf{ES}}$ of the ECMA-262 specification to increase the analysis precision. First, we extend abstract domains to support generalized occurrence typing with user-defined target types (§2.1) and define the abstract semantics for extended type analysis (§2.2), and prove its soundness (§2.3). Then, we define provenances of type refinements as their explanation and extend the abstract domain to track type propositions that influence the refinement during the type analysis (§2.4).

2.1 Extended Abstract Domain

Figure 7 shows the extended abstract domains for type analysis with occurrence typing. We first introduce three new components with symbols: 1) *type propositions* to represent restrictions on types of variables and symbols, 2) *symbolic types* for type aliasing, and 3) *type guards* to enable chained application of type propositions. Using these components, we extend the abstract domain to support the occurrence typing with user-defined target types as highlighted in green. In the extended abstract domain, a typing result $\Gamma \in \mathbb{\Gamma}$ is augmented with a type proposition $\psi \in \Psi$ that always holds at the current point in the specification. An abstract value $\widehat{v} \in \widehat{\mathbb{V}}$ is not just a type but a symbolic type $\tau^{\omega} \in \mathcal{T}^{\omega}$ with a corresponding type guard $G \in \mathbb{G}$. We explain details of each component in the rest of this section.

Type Propositions. A *type proposition* $\psi \in \Psi$ is a proposition on the types of variables and symbols. A *symbol* (or a *type variable*) $\omega \in \Omega$ represents an argument type flowed from callers; ω^{0}, ω^{1}, ... denote the types of the first, second, and so on arguments. The top ($\top_{\psi}$) and bottom ($\bot_{\psi}$) type propositions always hold and never hold, respectively. For example, a type proposition $\psi = (x : \mathbf{N}) \wedge (\omega^{0} <: \mathbf{S})$ denotes that the variable x is the number type $\mathbf{N}$ and the type of the

$$\frac{}{\lceil \tau \rceil_\psi = \tau} \qquad \frac{\psi \Rightarrow (\omega <: \tau)}{\lceil \omega \rceil_\psi = \tau} \qquad \frac{\lceil \tau^\omega \rceil_\psi = \tau}{\lceil \tau^\omega.\mathtt{f} \rceil_\psi = \tau.\mathtt{f}} \qquad \frac{\overline{\lceil \tau_k^\omega \rceil_\psi = \tau_k}}{\lceil \{\overline{s_k : \tau_k^\omega}\} \rceil_\psi = \{\overline{s_k : \tau_k}\}}$$

(a) Upper type bound $\lceil \tau^\omega \rceil_\psi$ operations on symbolic types τ^ω with a type proposition ψ.

$$\tau_1^\omega|_{\psi_1} \sqsubseteq \tau_2^\omega|_{\psi_2} \Leftrightarrow \begin{cases} \lceil \tau_1^\omega \rceil_{\psi_1} <: \tau_2 & \text{if } \tau_2^\omega = \tau_2 \\ \tau_1^\omega = \tau_2^\omega & \text{otherwise} \end{cases} \qquad \tau_1^\omega|_{\psi_1} \sqcup \tau_2^\omega|_{\psi_2} = \begin{cases} \tau_1^\omega & \text{if } \tau_1^\omega = \tau_2^\omega \\ \lceil \tau_1^\omega \rceil_{\psi_1} \sqcup \lceil \tau_2^\omega \rceil_{\psi_2} & \text{otherwise} \end{cases}$$

$$\tau_1^\omega|_{\psi_1} \sqcap \tau_2^\omega|_{\psi_2} = \begin{cases} \tau_1^\omega & \text{if } \tau_1^\omega \sqsubseteq \tau_2^\omega \\ \tau_2^\omega & \text{if } \tau_2^\omega \sqsubseteq \tau_1^\omega \\ \lceil \tau_1^\omega \rceil_{\psi_1} \sqcap \lceil \tau_2^\omega \rceil_{\psi_2} & \text{otherwise} \end{cases} \qquad \tau_1^\omega|_{\psi_1} \downarrow \tau_2 = \begin{cases} \tau_1^\omega & \text{if } \tau_1^\omega \sqsubseteq \tau_2^\omega \\ \lceil \tau_1^\omega \rceil_{\psi_1} \sqcap \tau_2^\omega & \text{otherwise} \end{cases}$$

(b) Partial order ($\sqsubseteq$), join ($\sqcup$), meet ($\sqcap$), and refinement ($\downarrow$) for symbolic types τ^ω with type propositions.

Fig. 8. Operations on symbolic types $\tau^\omega \in \mathcal{T}^\omega$.

first argument is a subtype of the string type **S**. We use only conjunctions of type propositions on variables or symbols for simplicity and efficiency of analysis in this paper, but other operations (e.g., disjunctions) or other kinds of type propositions can be freely added. While type propositions can be inferred basically from a dynamic type check expression $e : \tau$, they can be inferred from user-defined rules for specific expressions as well. We can refine types of variables and symbols using type propositions inferred during the analysis. Since the equality relation is crucial for type refinement, we represent aliases between types using symbolic types.

Symbolic Types. A *symbolic type* $\tau^\omega \in \mathcal{T}^\omega$ is either a concrete τ, a symbol ω, a symbolic field access $\tau^\omega.\mathtt{f}$, or a symbolic record type $\{\overline{\mathtt{f} : \tau^\omega}\}$. Symbolic types that share the same symbols can be refined together using the type propositions on the symbols. For example, two variables in the following abstract environment $\widehat{\rho} \in \widehat{\mathbb{E}}$ are refined together when the type proposition ψ_1 is refined into ψ_2 because they share the same symbol ω:

$$\widehat{\rho} = [\ x \mapsto \omega,\ y \mapsto \{\mathtt{f} : \omega\}\ ] \quad \text{means} \quad \begin{cases} [\ x \mapsto \mathbf{N} \sqcup \mathbf{S},\quad y \mapsto \{\mathtt{f} : \mathbf{N} \sqcup \mathbf{S}\}\ ] & \text{if } \psi_1 = \omega <: \mathbf{N} \sqcup \mathbf{S} \\ [\ x \mapsto \mathbf{N},\quad y \mapsto \{\mathtt{f} : \mathbf{N}\}\ ] & \text{if } \psi_2 = \omega <: \mathbf{N} \end{cases}$$

In addition, parametric polymorphism is supported by instantiating symbols inside symbolic types with concrete actual types at the call-sites. Figure 8a shows the operations on symbolic types. The upper type bound $\lceil \tau^\omega \rceil_\psi$ of a symbolic type τ^ω is defined with the current type proposition ψ. The lattice operations ($\sqsubseteq$, $\sqcup$, and $\sqcap$) on symbolic types are defined based on it. The refinement operation ($\downarrow$) on symbolic types is the key operation used in the type refinement in the extended type analysis. Note that $\lceil \tau^\omega \rceil_\psi$ is the upper type bound not the actual type of the symbolic type τ^ω because they can be instantiated with any concrete types satisfying the type propositions ψ. It means that the actual type of a symbolic type τ^ω is not fixed until the type propositions are applied to it.

Type Guards. A *type guard* $G \in \mathbb{G}$ is a mapping from user-defined target types $\mathcal{T}^T$ to type propositions Ψ. A single mapping $\tau^T \Rightarrow \psi$ in a type guard states that if a value is a target type τ^T, then the type proposition ψ always holds. Type guards allow additional type propositions to be applied sequentially, thereby enabling chained application of multiple propositions. For example,

Type Guard Lookup	$G(\tau) = \bigsqcap \{\psi \mid (\tau^T \Rightarrow \psi) \in G \wedge (\tau <: \tau^T)\}$
Type Guard Update	$G[\tau \Rightarrow \psi] = \lambda \tau^T. \begin{cases} G(\tau^T) \sqcap \psi & \text{if } \tau <: \tau^T \\ G(\tau^T) & \text{otherwise} \end{cases}$
Type Guard Refinement	$G \downarrow \tau = \lambda \tau^T. \begin{cases} \bot_\psi & \text{if } \tau \sqcap \tau^T = \bot_\tau \\ G(\tau^T) & \text{otherwise} \end{cases}$
Type Guard Field Lookup	$G.\mathtt{f} = \lambda \tau^T.\ G(\{\mathtt{f} : \tau^T\})$
Record Type Guard	$\{\overline{\mathtt{f}_j : G_j}\} = \lambda \tau^T.\ \bigwedge_j G_j(\tau^T.\mathtt{f}_j)$

Fig. 9. Operations on type guards $G \in \mathbb{G}$.

consider an abstract environment $\widehat{\rho}$ and its updated one $\widehat{\rho}'$ with a type proposition $\psi_1 = (x : \mathbf{N})$:

$$\widehat{\rho} = [\ x \mapsto (\mathbf{B} \sqcup \mathbf{N})\ [\mathbf{N} \Rightarrow \overline{(y : \mathbf{S})}^{\psi_2}],\quad y \mapsto \mathbf{B} \sqcup \mathbf{S}\ ] \quad \xrightarrow{\text{applying } \overline{(x : \mathbf{N})}^{\psi_1}} \quad [\ x \mapsto \mathbf{N}\ [\ldots],\quad y \mapsto \mathbf{S}\ ]$$

Then, the type of x is refined from $\mathbf{B} \sqcup \mathbf{N}$ to $\mathbf{N}$, which is a subtype of the target type $\mathbf{N}$ in the type guard for x. Therefore, another type proposition $\psi_2 = (y : \mathbf{S})$ in the guard also holds and refines the type of y to from $\mathbf{B} \sqcup \mathbf{S}$ to $\mathbf{S}$. Finally, two applied type propositions $\psi_1 = (x : \mathbf{N})$ and $\psi_2 = (y : \mathbf{S})$ are merged into the type proposition ψ' of the updated typing result $\Gamma' = (\widehat{\rho}', \psi', \delta')$. The traditional occurrence typing only supports two boolean literals, `#t` and `#f`, as target types. However, users can freely specify any types (even overlapping types) as target types $\mathcal{T}^T$ in occurrence typing in our extended type analysis.

The lattice operations ($\sqsubseteq$, $\sqcup$, and $\sqcap$) on type guards are defined in a point-wise manner, and other operations on type guards are defined in Figure 9. The guard lookup $G(\tau)$ and update $G[\tau \Rightarrow \psi]$ operations lookup and update propositions associated with target types that are supertypes of the given type τ. The guard refinement $G \downarrow \tau$ refines the type guard to remove impossible propositions for a given type τ. The field lookup $G.\mathtt{f}$ and record $\{\overline{\mathtt{f} : G}\}$ operations produce a new type guard for the result of a field lookup and a record allocation, respectively. Both are defined in a *query form*: for each target type τ^T of the result, they ask which proposition the original guard promises. If the field value $h(a)(\mathtt{f})$ belongs to τ^T, the base record a belongs to the open record type $\{\mathtt{f} : \tau^T\}$, so $G.\mathtt{f}$ looks up the base guard G with this record type. Dually, if an allocated record belongs to τ^T, each field value belongs to $\tau^T.\mathtt{f}_j$, so the record guard conjoins the lookups of the field guards. Note that a promise for a target type with multiple fields (e.g., $\{\mathtt{f} : \mathbf{N}, \mathtt{g} : \mathbf{S}\}$) cannot be recovered from a single field value; the field lookup soundly drops such a promise. Field updates do not have a dedicated guard operation; they are handled by effect-based weakening (Figure 14). For example, consider the following type guard G with five target types $\mathcal{T}^T = \{\mathtt{\#t}, \mathtt{\#f}, \mathbf{B}, \mathbf{N}, \{\mathtt{f} : \mathbf{N}\}\}$:

$$G = [\ \mathtt{\#t} \Rightarrow \psi_{\mathtt{\#t}},\quad \mathtt{\#f} \Rightarrow \psi_{\mathtt{\#f}},\quad \mathbf{B} \Rightarrow \psi_{\mathbf{B}},\quad \mathbf{N} \Rightarrow \psi_{\mathbf{N}}\quad \{\mathtt{f} : \mathbf{N}\} \Rightarrow \psi_{\mathtt{f}:\mathbf{N}}\ ]$$

Then, the result of each operation on the type guard G is as follows:

$$\begin{array}{lll}
G(\mathtt{\#t}) & = \psi_{\mathtt{\#t}} \sqcap \psi_{\mathbf{B}} & \\
G[\mathtt{\#t} \Rightarrow \psi] & = [\ \mathtt{\#t} \Rightarrow \psi_{\mathtt{\#t}} \sqcap \psi, \mathtt{\#f} \Rightarrow \psi_{\mathtt{\#f}}, \mathbf{B} \Rightarrow \psi_{\mathbf{B}} \sqcap \psi, \mathbf{N} \Rightarrow \psi_{\mathbf{N}} & \{\mathtt{f} : \mathbf{N}\} \Rightarrow \psi_{\mathtt{f}:\mathbf{N}}\ ] \\
G \downarrow \mathbf{B} & = [\ \mathtt{\#t} \Rightarrow \psi_{\mathtt{\#t}}, \mathtt{\#f} \Rightarrow \psi_{\mathtt{\#f}}, \mathbf{B} \Rightarrow \psi_{\mathbf{B}}, \mathbf{N} \Rightarrow \bot_\psi & \{\mathtt{f} : \mathbf{N}\} \Rightarrow \bot_\psi\ ] \\
G.\mathtt{f} & = [\ \mathtt{\#t} \Rightarrow \top_\psi, \mathtt{\#f} \Rightarrow \top_\psi, \mathbf{B} \Rightarrow \top_\psi, \mathbf{N} \Rightarrow \psi_{\mathtt{f}:\mathbf{N}} & \{\mathtt{f} : \mathbf{N}\} \Rightarrow \top_\psi\ ] \\
\{\mathtt{f} : G\} & = [\ \mathtt{\#t} \Rightarrow \top_\psi, \mathtt{\#f} \Rightarrow \top_\psi, \mathbf{B} \Rightarrow \top_\psi, \mathbf{N} \Rightarrow \top_\psi & \{\mathtt{f} : \mathbf{N}\} \Rightarrow \psi\ ]
\end{array}$$

$$\widehat{[\![-]\!]}_i : (\mathcal{L} \times \widehat{\Sigma}) \to \widehat{\Sigma}$$

$$[\![x = \{\overline{\mathtt{f}_k : e_k}\}]\!]_i(l, \widehat{\sigma}) = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) \quad \text{where } \Gamma_1 = \Gamma.\widehat{\mathsf{update}}_x(x, \{\overline{\mathtt{f}_k : \widehat{[\![e_k]\!]}_e(\Gamma)}\}.\ \widehat{\mathsf{bind}}(\psi))$$

$$[\![\mathtt{call}\ \overline{x = e_0(\overline{e_k})}]\!]_i(l, \widehat{\sigma}) = (\widehat{\kappa}_1, \{\overline{l_j \mapsto \Gamma_j}\}) \quad \text{where } \begin{cases} \widehat{v}_0 & = \widehat{[\![e_0]\!]}_e(\Gamma) \\ (\tau_k^\omega, _) & = \widehat{[\![e_k]\!]}_e(\Gamma) \\ \psi' & = \bigwedge\{\omega^{\mathtt{k}} <: \lceil \tau_k^\omega \rceil_\psi\} \\ \{\Lambda_j\} & = \{\overline{\lambda(\overline{x_{j,k}}).l_j}\} = \gamma_v(\widehat{v}_0) \cap \mathcal{F} \\ \Gamma_j & = (\{x_{j,k} \mapsto \omega^{\mathtt{k}}\}, \psi', \bot_\delta) \\ \widehat{\kappa}_1 & = \{\Lambda_j \mapsto (l, x)\} \end{cases}$$

$$[\![\overline{\mathtt{return}\ e}]\!]_i(l, \widehat{\sigma}) = (\varnothing, \{\overline{\mathsf{next}(l_k) \mapsto \Gamma'_k}\}) \quad \text{where } \begin{cases} \{\overline{(l_k, x_k)}\} & = \widehat{\kappa} \circ \mathsf{func}(l) \\ \mathsf{inst}(l_k) & = \mathtt{call}\ x_k = _(\overline{e_{k,j}}) \\ \Gamma_k & = \widehat{\xi}(l_k).\widehat{\mathsf{apply}}(\delta) \\ \widehat{v} & = \widehat{[\![e]\!]}_e(\Gamma) \\ \widehat{v}'_k & = \widehat{v}.\ \mathsf{instantiate}(\widehat{\xi}(l_k), \overline{e_{k,j}}, \delta) \\ \widehat{v}_k & = \widehat{v}'_k.\ \widehat{\mathsf{bind}}(\Gamma_k.\psi) \\ \Gamma'_k & = \Gamma_k.\widehat{\mathsf{update}}_x(x_k, \widehat{v}_k) \end{cases}$$

$$\widehat{[\![-]\!]}_e : \mathbb{\Gamma} \to \widehat{\mathbb{V}}$$

$$\widehat{[\![e]\!]}_e(\Gamma) = \ldots [\ G\ ].\ \widehat{\mathsf{bind}}(\psi) \ \text{ where } \ G = \widehat{\mathsf{infer}}_G(\Gamma, e)$$

Fig. 10. Extended abstract semantics of instructions and expressions where $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ and $\xi(l) = \Gamma = (\widehat{\rho}, \psi, \delta)$.

When a type guard G is bound to a specific symbolic type τ^ω in an abstract value (i.e., $\widehat{v} = \tau^\omega[G]$)), the refinement operation is used for removing infeasible propositions (i.e., $G \downarrow \lceil \tau^\omega \rceil_\psi$). In addition, we can reduce the size of the type guard by skipping meaningless propositions that always hold or never hold according to the context. If the current typing result is $\Gamma = (\widehat{\rho}, \psi, \delta)$ and a skipped proposition ψ' is bound to a target type τ^T for the symbolic type τ^ω in an abstract value, then ψ' is $\bot_\psi$ if $\lceil \tau^\omega \rceil_\psi \sqcap \tau^T = \bot_\psi$ or ψ otherwise. For example, consider the following abstract value $\widehat{v}$ with three target types $\mathcal{T}^T = \{\mathbf{N}, \mathtt{\#t}, \mathtt{\#f}\}$:

$$\widehat{v} = \mathbf{B}\ [\ \mathtt{\#f} \Rightarrow \psi_{\mathtt{\#f}}\ ] \quad \text{means} \quad \mathbf{B}\ [\ \mathbf{N} \Rightarrow \bot_\psi \quad \mathtt{\#t} \Rightarrow \psi, \quad \mathtt{\#f} \Rightarrow \psi_{\mathtt{\#f}}\ ]$$

Then, the skipped propositions associated with target types $\mathbf{N}$ and $\mathtt{\#t}$ are $\bot_\psi$ and ψ, respectively, because $\mathbf{B} \sqcap \mathbf{N} = \bot_\psi$ but $\mathbf{B} \sqcap \mathtt{\#t} = \mathtt{\#t} \neq \bot_\psi$. If an abstract value $\widehat{v} = \tau^\omega[G]$ having a top type guard $G = \top_G$, we simply write τ^ω instead of $\tau^\omega[\top_G]$. The lattice operations on other abstract domains are defined as same as the original abstract domain in §1.2 or extended in a point-wise manner. For example, $\{\overline{\mathtt{f}_k : \widehat{v}_k}\} = \{\overline{\mathtt{f}_k : \tau_k^\omega}\}\ [\{\overline{\mathtt{f}_k : G_k}\}]$ where $\widehat{v}_k = \tau_k^\omega[G_k]$.

2.2 Extended Abstract Semantics

This section describes the abstract semantics for extended type analysis with occurrence typing as shown in Figure 6. We highlight the extended or modified parts in green. Type propositions are 1) **inferred** when analyzing expressions or producing abstract values, 2) **propagated** into other program points even across functions, 3) used to **refine** types of variables and symbols in branch or assertion instructions, and 4) **weakened** when variables or fields are mutated.

$$\widehat{\mathsf{infer}}_G(\Gamma, e : \tau) = \top_G[\#\mathtt{t} \Rightarrow \psi_{\#\mathtt{t}}][\#\mathtt{f} \Rightarrow \psi_{\#\mathtt{f}}] \quad \text{where} \begin{cases} (\tau^\omega, _) = \widehat{[\![e]\!]}_e(\Gamma) \\ \psi_{\#\mathtt{t}} = \widehat{\mathsf{infer}}_\psi(e, \tau^\omega)(\lceil \tau^\omega \rceil_\psi \sqcap \tau) \\ \psi_{\#\mathtt{f}} = \widehat{\mathsf{infer}}_\psi(e, \tau^\omega)(\lceil \tau^\omega \rceil_\psi \setminus \tau) \end{cases}$$
$$\widehat{\mathsf{infer}}_G(\Gamma, e) = \top_G \quad \text{otherwise}$$

(a) The type guard inference operation $\widehat{\mathsf{infer}}_G : (\mathbb{\Gamma} \times \mathcal{E}) \to \mathbb{G}$ where $\Gamma = (\widehat{\rho}, \psi, \delta)$.

$$\widehat{\mathsf{infer}}_\psi(e, \tau^\omega)(\tau) = \begin{cases} \psi & \text{if } \tau^\omega \in \mathcal{T} \wedge (e <:? \tau \Downarrow \psi) \\ \psi & \text{if } \tau^\omega \notin \mathcal{T} \wedge (\tau^\omega <:? \tau \Downarrow \psi) \\ \top_\psi & \text{otherwise} \end{cases}$$

where $e <:? \tau \Downarrow \psi$ or $\tau^\omega <:? \tau \Downarrow \psi$ is defined as follows:

$$\frac{}{x <:? \tau \Downarrow x : \tau} \quad \frac{e <:? \{\mathtt{f} : \tau\} \Downarrow \psi}{e.\mathtt{f} <:? \tau \Downarrow \psi} \quad \frac{}{\omega <:? \tau \Downarrow \omega <: \tau} \quad \frac{\tau^\omega <:? \{\mathtt{f} : \tau\} \Downarrow \psi}{\tau^\omega.\mathtt{f} <:? \tau \Downarrow \psi} \quad \frac{\overline{\tau_k^\omega <:? \tau.\mathtt{f}_k \Downarrow \psi_k}}{\{\mathtt{f}_k : \tau_k^\omega\} <:? \tau \Downarrow \bigwedge\{\overline{\psi_k}\}}$$

(b) The type proposition inference operation $\widehat{\mathsf{infer}}_\psi : (\mathcal{E} \times \mathcal{T}^\omega) \to \mathcal{T} \to \Psi$.

$$\tau^\omega[G].\widehat{\mathsf{bind}}(\psi) = \tau^\omega[G'] \text{ where } G' = G[\lceil \tau^\omega \rceil_\psi \Rightarrow \psi]$$

(c) The binding operation $\widehat{v}.\widehat{\mathsf{bind}} : \Psi \to \mathbb{G}$ that binds type propositions to target types in type guards.

Fig. 11. Operations related to **type proposition inference**.

Type Proposition Inference. As shown in Figure 6, the part for symbolic types in the abstract semantics for expressions is same as the original abstract semantics with the corresponding operations on symbolic types instead of concrete types. The main difference is that type propositions in the type guard for the expression is inferred during the analysis using the $\widehat{\mathsf{infer}}_G$ operation defined in Figure 11a. The $\widehat{\mathsf{infer}}_G$ operation basically infers a type guard G for a dynamic type check expression $e : \tau$. It first analyzes the expression e to get its symbolic type τ^ω. Then, it infers two type propositions $\psi_{\#\mathtt{t}}$ and $\psi_{\#\mathtt{f}}$ holding when e is a subtype of τ and a disjoint type from τ, respectively, using the $\widehat{\mathsf{infer}}_\psi$ operation defined in Figure 11b. The inferred type propositions $\psi_{\#\mathtt{t}}$ and $\psi_{\#\mathtt{f}}$ are then bound to the target types `#t` and `#f` in the type guard G, respectively. While this formalization only supports type guards for dynamic type check expressions, we can define additional rules for specific expressions to infer type guards and will define them in §5 of the paper. Additionally, type guards can be inferred when producing abstract values using $\widehat{\mathsf{bind}}$ operation defined in Figure 11. It checks the upper type of the produced abstract value $\widehat{v}$ is a subtype of a target type τ^T and binds the type proposition ψ to the target type τ^T in the type guard G. The $\widehat{\mathsf{bind}}$ operation is called whenever an abstract value is produced (e.g., results of expressions, allocated records, returned values, etc.) in the abstract semantics.

Type Proposition Propagation. The inferred type propositions are propagated into other program points even across functions stored in type guards of abstract values. When calling a function, analysis introduces indexed symbols ω^{k} to represent type variables for the actual types of the arguments. The inferred type propositions for the arguments inside the function are propagated into the call-site stored in the type guard of the returned abstract values. Then, the type propositions are instantiated with the argument expressions and their actual abstract values using the instantiate

$$\tau^\omega[G].\mathsf{instantiate}(\Gamma, \overline{e_k}, \delta) = \tau_0^\omega[G_0 \sqcap G'] \text{ where } \begin{cases} \widehat{v}_k &= \tau_k^\omega[G_k] = \widehat{[\![e_k]\!]}_e(\Gamma) \\ \tilde{\tau}_k^\omega &= \mathsf{weaken}_\delta(\tau_k^\omega) \\ \tau_0^\omega &= \tau^\omega[\overline{\omega^{\mathsf{k}} \mapsto \tilde{\tau}_k^\omega}] \\ G_0 &= \tau^\omega[\omega^{\mathsf{k}} \mapsto \mathsf{weaken}_\delta(G_k)] \\ G' &= \lambda\tau^T.G(\tau^T).\mathsf{instantiate}_\delta([\overline{(e_k, \tilde{\tau}_k^\omega)}]) \end{cases}$$

$$\text{and } \psi.\mathsf{instantiate}_\delta([\overline{(e_k, \tilde{\tau}_k^\omega)}]) = \begin{cases} \bot_\psi & \text{if } \psi = \bot_\psi \\ \psi_0.\mathsf{instantiate}_\delta(\cdots) \wedge \psi_1.\mathsf{instantiate}_\delta(\cdots) & \text{if } \psi = \psi_0 \wedge \psi_1 \\ \mathsf{weaken}_\delta(\widehat{\mathsf{infer}}_\psi(e_k, \tilde{\tau}_k^\omega)(\tau)) & \text{if } \psi = \omega^{\mathsf{k}} <: \tau \\ \top_\psi & \text{otherwise} \end{cases}$$

Fig. 12. Instantiation $\widehat{v}.\mathsf{instantiate} : (\mathbb{\Gamma} \times \mathcal{E} \times \Delta) \to \widehat{\mathbb{V}}$ of abstract values with argument expressions and the callee effect for inter-procedural **type proposition propagation**.

operation defined in Figure 12. When returning from a function (Figure 10), the typing result $\xi(l_k)$ of the caller at the call site is weakened with the callee effect δ to obtain Γ_k, which describes what the caller still knows after the call. The returned abstract value is instantiated with the argument expressions evaluated in $\xi(l_k)$ rather than in Γ_k, because each symbol ω^{k} denotes the value of the k-th argument at the call; the changes since the call are taken into account by the callee effect δ in the instantiation as explained below. Finally, the instantiated value is bound with the proposition $\Gamma_k.\psi$ of the caller rather than that of the callee, which may mention local variables of the callee. The callee may mutate the heap before it returns, and the instantiation takes its effect δ into account. A proposition $\omega^{\mathsf{k}} <: \tau$ inferred in the callee is a fact about the *value* of the k-th argument at the call, which never changes. However, its translation $\widehat{\mathsf{infer}}_\psi(e_k, \tilde{\tau}_k^\omega)(\tau)$ is a proposition about the argument *expression* e_k in the caller, which is evaluated in the heap at the return. Thus, we weaken the translated proposition with the callee effect δ using weaken_δ defined in Figure 14. For the same reason, the symbolic types τ_k^ω and the type guards G_k of the arguments are weakened with weaken_δ; for example, a field lookup $\tau^\omega.\mathtt{f}$ is replaced with its upper type bound if the callee writes the field `f`, because it may denote a different value at the return. For example, consider the following callee g and its call:

$$\mathsf{g(a, o) = b = (a : N);\ o.f = "abc";\ return\ b} \qquad\qquad \mathsf{r = g}(x.\mathtt{f}, x)$$

at a call site l_k where $\widehat{[\![x.\mathtt{f}]\!]}_e(\xi(l_k)) = \mathbf{N}$ before the callee overwrites $x.\mathtt{f}$. The returned abstract value is $\widehat{v}_{\mathsf{ret}} = \mathbf{B}[\#\mathtt{t} \Rightarrow \omega^\emptyset <: \mathbf{N}]$ with the callee effect $\delta = [\mathtt{f} \mapsto \mathbf{S}]$. The instantiation translates $\omega^\emptyset <: \mathbf{N}$ into $x : \{\mathtt{f} : \mathbf{N}\}$ using $\widehat{\mathsf{infer}}_\psi$ and weakens it with δ:

$$\widehat{v}_{\mathsf{ret}}.\mathsf{instantiate}(\xi(l_k), [x.\mathtt{f}, x], \delta) = \mathbf{B}[\#\mathtt{t} \Rightarrow x : \{\mathtt{f} : \mathbf{N} \sqcup \mathbf{S}\}]$$

Without the weakening, the guard would claim that $x.\mathtt{f}$ is a number when `r` is `#t`, which is wrong since the callee overwrites $x.\mathtt{f}$ with a string after the dynamic type check. In general, both type guards 1) G_0 existing in the argument abstract values and 2) G' inferred in the callee are propagated and merged into the type guard of the instantiated returned abstract value.

Type Refinement. Unlike syntactic type refinement in the original type analysis, the type refinement utilizes the inferred type propositions in the type guard to refine the type of variables and symbols in branch or assertion instructions. If a proposition $x : \tau$ for a variable x is applicable, the abstract value of x is refined as $\widehat{\rho}(x)|_\psi \downarrow \tau$ using the refinement operation ($\downarrow$) for symbolic types defined in Figure 8b and for type guards defined in Figure 9. In addition, it enables chained

$$\boxed{\Gamma.\widehat{\mathsf{refine}}(e, b) = \Gamma.\ \widehat{\mathsf{refine}}(\psi) \text{ where } _[G] = \widehat{[\![e]\!]}_e(\Gamma) \text{ and } \psi = G[b]}$$

$$\text{and } \Gamma.\widehat{\mathsf{refine}}(\psi') = \begin{cases} (\widehat{\rho}[x \mapsto \widehat{v}], \psi \wedge \psi', \delta).\widehat{\mathsf{refine}}(G[\tau]) & \text{if } (\psi' = x : \tau) \wedge \widehat{v} = (\widehat{\rho}(x)|_\psi \downarrow \tau) \\ \Gamma.\widehat{\mathsf{refine}}(\psi_0) \sqcap \Gamma.\widehat{\mathsf{refine}}(\psi_1) & \text{if } \psi' = (\psi_0 \wedge \psi_1) \\ (\widehat{\rho}, \psi \wedge \psi', \delta) & \text{otherwise} \end{cases}$$

Fig. 13. Modified **type refinement** operation $\Gamma.\widehat{\mathsf{refine}} : \Psi \to \mathbb{\Gamma}$ where $\Gamma = (\widehat{\rho}, \psi, \delta)$.

application of the type propositions in the type guard as explained in §2.1 by recursively invoking the $\widehat{\mathsf{refine}}$ operation if the result of type refinement satisfies another target type $\tau^T \in \mathcal{T}^T$.

Weakening Type Propositions. Since our target language allows mutation of variables and fields, we need to weaken the type propositions in the type guard when they are mutated for sound type analysis. As shown in Figure 14, we can weaken the type propositions using the modified variable and field update operations $\Gamma.\widehat{\mathsf{update}}_x$ and $\Gamma.\widehat{\mathsf{update}}_\mathtt{f}$. After mutating a variable x with a new abstract value $\widehat{v}$, any propositions associated with x are invalid and should be removed from the type guard and typing result. The $\mathsf{weaken}_x(x)$ operation iterates over all type propositions in the current typing result Γ and invalidates the propositions associated with the variable x. On the other hand, a field update and a function return weaken the typing result with an effect δ using the $\Gamma.\widehat{\mathsf{apply}}(\delta)$ operation. It weakens every type, proposition, and type guard in Γ with weaken_δ defined in Figure 14 and accumulates the effect. The weaken_δ operation extends the type of each field $\mathtt{f} \in \mathrm{dom}(\delta)$ in every record type with $\delta(\mathtt{f})$, including nested record types. For example, the type of the field $\mathtt{f}$ is weakened from $\mathbf{N}$ to $\mathbf{N} \sqcup \mathbf{B}$ with the effect $\delta = [\mathtt{f} \mapsto \mathbf{B}]$:

$$\omega <: \{\mathtt{f} : \mathbf{N}, \quad \mathtt{g} : \mathbf{S}\} \xrightarrow{\mathsf{weaken}_\delta} \omega <: \{\mathtt{f} : (\mathbf{N} \sqcup \mathbf{B}), \quad \mathtt{g} : \mathbf{S}\}$$

Since the membership of a heap-dependent type (i.e., a type containing record types) can change after mutation, weaken_δ conservatively drops such types in negative positions (the right-hand side τ_2 of a difference $\tau_1 \setminus \tau_2$) and drops the entries of type guards for heap-dependent target types. Heap-independent target types used in practice (e.g., $\#\mathtt{t}$, $\#\mathtt{f}$, and completion types) are not affected by this restriction. For a symbolic type, a symbol ω denotes the value of an argument and is stable, but a field lookup $\tau^\omega.\mathtt{f}$ with $\mathtt{f} \in \mathrm{dom}(\delta)$ may denote a different value after mutation, so it is replaced with its upper type bound. The same operation $\widehat{\mathsf{apply}}$ is used to apply the callee effect to the typing result of the caller when returning from a function (Figure 10) and the callee effect also weakens the propagated type propositions in the instantiation (Figure 12). While this formalization uses uniform abstract addresses for all record types – potentially introducing imprecision – we use partitions abstract addresses to avoid such imprecision in §5 of the paper.

$$\Gamma.\widehat{\mathsf{update}}_x(x, \widehat{v}) = (\widehat{\rho}[x \mapsto \widehat{v}], \psi, \delta).\ \mathsf{weaken}_x(x)$$

(a) Modified variable update operation $\Gamma.\widehat{\mathsf{update}}_x : (\mathcal{X} \times \widehat{\mathbb{V}}) \to \mathbb{\Gamma}$.

$$\begin{aligned}\Gamma.\widehat{\mathsf{update}}_{\mathsf{f}}(\widehat{v}_1, \mathsf{f}, \widehat{v}_2) &= \Gamma.\ \widehat{\mathsf{apply}}([\mathsf{f} \mapsto \tau_2]) \\ \Gamma.\widehat{\mathsf{apply}}(\delta') &= (\{\overline{x \mapsto \mathsf{weaken}_{\delta'}(\widehat{v})}\}, \mathsf{weaken}_{\delta'}(\psi), \delta \fatsemi \delta') \ \text{ where } \widehat{\rho} = \{\overline{x \mapsto \widehat{v}}\}\end{aligned}$$

(b) Modified field update and effect application operations where $\widehat{v}_2 = \tau_2^{\omega}[_]$, $\tau_2 = \mathsf{weaken}_{[\mathsf{f} \mapsto \top]}(\lceil \tau_2^{\omega} \rceil_{\psi})$, and $(\delta \fatsemi \delta')(\mathsf{f}) = \mathsf{weaken}_{\delta'}(\delta(\mathsf{f})) \sqcup \delta'(\mathsf{f})$ (an undefined side is omitted).

$$\begin{aligned}
\mathsf{weaken}_{\delta}(\{\overline{\mathsf{f}_k : \tau_k}\}) &= \{\overline{\mathsf{f}_k : \mathsf{weaken}_{\delta}(\tau_k) \sqcup \delta(\mathsf{f}_k)}\} & \\
\mathsf{weaken}_{\delta}(\tau_1 \setminus \tau_2) &= \mathsf{weaken}_{\delta}(\tau_1) \setminus \tau_2 & \text{if } \tau_2 \text{ has no record types, else } \mathsf{weaken}_{\delta}(\tau_1) \\
\mathsf{weaken}_{\delta}(G) &= \lambda \tau^T.\ \mathsf{weaken}_{\delta}(G(\tau^T)) & \text{if } \tau^T \text{ has no record types, else } \top_{\psi} \\
\mathsf{weaken}_{\delta}(\tau^{\omega}.\mathsf{f}) &= \mathsf{weaken}_{\delta}(\lceil \tau^{\omega} \rceil_{\psi}.\mathsf{f}) & \text{if } \mathsf{f} \in \mathrm{dom}(\delta), \text{ else } \mathsf{weaken}_{\delta}(\tau^{\omega}).\mathsf{f} \\
\mathsf{weaken}_{\delta}(\{\overline{\mathsf{f}_k : \tau_k^{\omega}}\}) &= \{\overline{\mathsf{f}_k : \tau'^{\omega}_k}\} & \\
\tau'^{\omega}_k &= \mathsf{weaken}_{\delta}(\lceil \tau_k^{\omega} \rceil_{\psi}) \sqcup \delta(\mathsf{f}_k) & \text{if } \mathsf{f}_k \in \mathrm{dom}(\delta), \text{ else } \mathsf{weaken}_{\delta}(\tau_k^{\omega}) \\
\mathsf{weaken}_{\delta}(\tau^{\omega}[G]) &= \mathsf{weaken}_{\delta}(\tau^{\omega})[\mathsf{weaken}_{\delta}(G)] & \mathsf{weaken}_{\delta}(\omega) = \omega \\
\mathsf{weaken}_{\delta}(x : \tau) &= x : \mathsf{weaken}_{\delta}(\tau) & \mathsf{weaken}_{\delta}(\omega <: \tau) = \omega <: \mathsf{weaken}_{\delta}(\tau)
\end{aligned}$$

(c) Effect-based weakening weaken_{δ} where $\delta(\mathsf{f}) = \bot$ if $\mathsf{f} \notin \mathrm{dom}(\delta)$ and ψ is the current type proposition. It is homomorphic for $\sqcup$, $\sqcap$, and $\wedge$, and other types and propositions are unchanged.

Fig. 14. Modified variable/field update operations and effect-based weakening for **weakening type propositions** where $\Gamma = (\widehat{\rho}, \psi, \delta)$.

2.3 Soundness

We define the meaning of the extended abstract domain relative to a heap h, an environment ρ, and a *symbol valuation* $\eta : \Omega \to \mathbb{V}$ that maps each symbol ω^{k} to the value of the k-th argument when the current function was called. Since a symbol denotes an argument *value* rather than an argument expression, its meaning never changes while the callee runs. Type propositions, type guards, and abstract values are interpreted as follows:

$$(h, \rho, \eta) \models x : \tau \iff (h, \rho(x)) \in \gamma_\tau(\tau) \qquad (h, \rho, \eta) \models \omega <: \tau \iff (h, \eta(\omega)) \in \gamma_\tau(\tau)$$
$$(h, \rho, \eta, v) \models G \iff \forall \tau^T.\ (h, v) \in \gamma_\tau(\tau^T) \Rightarrow (h, \rho, \eta) \models G(\tau^T)$$
$$v \in \gamma^\eta_{h,\rho}(\tau^\omega[G]) \iff v \in [\![\tau^\omega]\!]^\eta_h \wedge (h, \rho, \eta, v) \models G$$

where $[\![\tau^\omega]\!]^\eta_h$ is the set of values described by the symbolic type τ^ω (e.g., $[\![\omega]\!]^\eta_h = \{\eta(\omega)\}$). Note that the target type membership and the proposition in a type guard are judged in the *same* heap h; this is why both type propositions and type guards must be weakened when the heap changes. A typing result $\Gamma = (\widehat{\rho}, \psi, \delta)$ covers a memory (h, ρ) with the entry heap h_0 of the current function if $\rho(x) \in \gamma^\eta_{h,\rho}(\widehat{\rho}(x))$ for all variables x, $(h, \rho, \eta) \models \psi$, and every field changed since h_0 belongs to $\mathrm{dom}(\delta)$ and holds a value of type $\delta(\mathtt{f})$. A concrete state is covered by an analysis result $\widehat{\sigma} = (\widehat{\kappa}, \xi)$, written $\sigma \in \gamma(\widehat{\sigma})$, if every frame in its call stack is covered by ξ at its label with its own symbol valuation and entry heap; the full definition is in the supplementary material.

Instead of the least fixpoint, we state soundness for any analysis result $\widehat{\sigma}$ that is a post-fixpoint of the abstract semantics, written $\mathsf{Valid}(P, \widehat{\sigma})$: for each instruction at a label l, its abstract semantics in Figure 10 applied to $\xi(l)$ is included in the typing results of its successors, including the entries of callees and the return sites of callers.

Theorem 2.1 (Soundness). *For a well-formed program P and an analysis result $\widehat{\sigma}$ with $\mathsf{Valid}(P, \widehat{\sigma})$, if $\sigma_0 \in \gamma(\widehat{\sigma})$ and $\sigma_0 \rightsquigarrow^* \sigma$, then $\sigma \in \gamma(\widehat{\sigma})$. In particular, if a variable x has a value v at a reachable state with a heap h and a label l, and $\xi(l) = (\widehat{\rho}, \psi, _)$ with $\widehat{\rho}(x) = \tau^\omega[_]$, then $(h, v) \in \gamma_\tau(\lceil \tau^\omega \rceil_\psi)$.*

Proof Sketch. The proof is by induction on the reachability using the preservation of $\gamma(\widehat{\sigma})$ by each concrete transition. The key lemma is *transport*: if a type, a type proposition, or a type guard holds in a heap h and the heap evolves into h' by writing only fields in $\mathrm{dom}(\delta)$ with values of type $\delta(\mathtt{f})$, then its weakening by weaken_δ holds in h'. Field updates directly use this lemma. The return case is the most involved one: it combines the transport lemma with the stability of the argument descriptions $\tilde{\tau}^\omega_k$ to show that the instantiated type guard (Figure 12) holds in the heap at the return. We mechanized the concrete semantics, the extended analysis, and the proof in Lean 4 without any admitted lemma (supplementary material). The theorem guarantees that the analysis over-approximates all reachable states, and we discuss its relation to the implementation in §7 of the paper.

2.4 Provenance Tracking

In this section, we introduce the provenance, which augments type propositions to add an explanation of why the type proposition can ensure that refinement is legal.

2.4.1 Definition. A *provenance* is bound with an *explanation* that shows why the refinement can happen; at the same time, it is an abstraction of the propagation path of propositions as a citizen of the abstract domain. It gives the simplest possible call paths set for each variable and each

$$\tau_{\zeta_1} \sqsubseteq \tau_{\zeta_2} \Leftrightarrow \text{if } \tau_1 \neq \tau_2 \text{ then } \tau_1 \sqsubseteq \tau_2 \text{ else } \begin{cases} \zeta_1.\text{child} \sqsubseteq \zeta_2.\text{child} & \text{if } \zeta_1 = \zeta_2 = \text{Root}(l, _) \mid \text{Call}(l, _) \mid \text{Join}(l, _) \mid \text{Meet}(l, _) \\ \texttt{\#t} & \text{if } \zeta_1 = \zeta_2 = \text{Leaf}(l) \\ \texttt{\#f} & \text{otherwise} \end{cases}$$

(a) Partial order $\sqsubseteq$: $\mathcal{T}_{\mathbb{Z}} \times \mathcal{T}_{\mathbb{Z}} \to \mathbb{B}$

$$\begin{aligned} \tau_{\zeta_1} \sqcup \tau_{\zeta_2} &= \text{canonical}\left(\begin{cases} \tau_{\zeta_1} & \text{if } \tau_{\zeta_2} \sqsubseteq \tau_{\zeta_1} \\ \tau_{\zeta_2} & \text{if } \tau_{\zeta_1} \sqsubseteq \tau_{\zeta_2} \\ (\tau_1 \sqcup \tau_2)_{(\zeta_1 \sqcup \zeta_2)} & \text{otherwise} \end{cases}\right) \\ \text{where } \zeta_1|_{\tau_1} \sqcup \zeta_2|_{\tau_2} &= \begin{cases} \zeta_1 & \text{if } \zeta_1 = \zeta_2 \\ \text{Join}(s_1 \sqcup s_2) & \text{if } \zeta_1 = \text{Join}(s_1) \wedge \zeta_2 = \text{Join}(s_2) \\ \text{Join}(s_1 \sqcup \{\tau_{\zeta_2}\}) & \text{if } \zeta_1 = \text{Join}(s_1) \wedge \zeta_2 = \text{Call} \mid \text{Leaf} \\ \text{Join}(s_2 \sqcup \{\tau_{\zeta_1}\}) & \text{if } \zeta_2 = \text{Join}(s_2) \wedge \zeta_1 = \text{Call} \mid \text{Leaf} \\ \text{Join}(\{\tau_{\zeta_1}, \tau_{\zeta_2}\}) & \text{otherwise} \end{cases} \end{aligned}$$

(b) Join operation $\sqcup : \mathcal{T}_{\mathbb{Z}} \times \mathcal{T}_{\mathbb{Z}} \to \mathcal{T}_{\mathbb{Z}}$

$$\begin{aligned} \text{canonical}(\text{Join}(\{\overline{\tau_\zeta}\})) &= \begin{cases} \text{canonical}(\text{Join}(\{\overline{\tau_\zeta}\} \setminus \{\tau_{\zeta_1}\})) \\ \quad \text{if } \exists \tau_{\zeta_2} \in \{\overline{\tau_\zeta}\} \setminus \{\tau_{\zeta_1}\} \text{ s.t. } (\tau_1 <: \tau_2 \wedge \tau_1 \neq \tau_2) \\ \text{Join}(\{\overline{\text{canonical}(\tau_\zeta)}\}) \quad \text{otherwise} \end{cases} \\ \text{canonical}(\tau_\zeta) &= \tau_\zeta \quad \text{for } \zeta \neq \text{Join}(_) \wedge \zeta \neq \text{Meet}(_) \end{aligned}$$

(c) Auxiliary operation canonical : $\mathcal{T}_{\mathbb{Z}} \to \mathcal{T}_{\mathbb{Z}}$

Fig. 15. Domain operations for types with provenance.

refinement, that affected the type guard used at the point.

$$\begin{aligned} &\textbf{Provenance} && \zeta \in \mathbb{Z} &&= \text{Nodes} \\ &\textbf{Type With Provenance} && \tau_\zeta \in \mathcal{T}_{\mathbb{Z}} &&= \mathcal{T} \times \mathbb{Z} \\ &\textbf{Type Propositions} && \psi \in \Psi &&= (\mathcal{X} \uplus \Omega) \to \mathcal{T}_{\mathbb{Z}} \end{aligned}$$

$$\text{Nodes} \ni \zeta = \text{Root}(l, \tau_\zeta) \mid \text{Call}(l, \tau_\zeta) \mid \text{Join}(l, \{\overline{\tau_\zeta}\}) \mid \text{Meet}(l, \{\overline{\tau_\zeta}\}) \mid \text{Leaf}(l) \mid \bot \mid \top$$

The type with provenance is same as the type in the section before, but augmented with the provenance tree. The provenance tree are made of multiple kinds of nodes represents the path in term of the analysis point, which means a set of locations we want to track the action (change of the type guard). Root, Call has a single refinement point with a single child, and Join, Meet has a set of children which can be retrieved by ζ.child. Most of definitions are trivial, but we note that Root means the refinement point and Leaf means the generation point of the guard.

2.4.2 Domain Operations. Provenances are not simple logs but a part of abstract domain. This means, if a pair of provenances are merged, we should demonstrate that a provenance represents the simplest path that can explain both provenances. We define here a operation between types with provenance as Figure 15, with a slight abuse of annotation with $\tau_{\zeta_i} = (\tau_i, \zeta_i)$. Since rules for $\bot$ and $\top$ are trivial, we omit them. Note that it omits a unnecessary nodes if a simpler explanation is possible, let the $\zeta_1 = (\mathbf{N}, \text{Call}(l_1))$ and $\zeta_2 = (\mathbf{N} \mid \mathbf{S}, \text{Call}(l_2))$, then $\zeta_1 \sqcup \zeta_2$ ignores the former since the latter is enough to explain the refinement. We do not explain each operation in detail due to the limit of space, but the essence of the provenance is stated in §6.3.2 of the paper.

3 Soundness of the Extended Type Analysis

This section defines the concretization of the extended abstract domain in §2 and states the soundness theorem of the extended type analysis with its proof structure. All definitions and the proof are mechanized in Lean 4 without any admitted lemma in the directory `proof/` of the artifact, whose `README.md` maps each definition and lemma to the Lean code.

Scope of the Mechanization. The mechanization covers the core of $\mathsf{IR}_{\mathsf{ES}}$: expressions (primitives, variables, field lookups, unary and binary operations, functions, and dynamic type checks) and instructions (assignments, record allocations, field updates, branches, assertions, calls, and returns). Unary and binary operators are uninterpreted, and the analysis approximates their results with $\top$. For branches, the false branch uses the target type $\top \backslash \#\mathtt{t}$ (non-true) instead of `#f`, following the truthiness of the concrete semantics.

3.1 Concrete States

A concrete state $\sigma = (\kappa, l, m)$ consists of a calling context κ, a label l, and a memory $m = (h, \rho)$. A calling context is either empty (ϵ) or the state of the caller $\sigma_c = (\kappa_c, l_c, m_c)$ at its call instruction at l_c. The *entry heap* of a frame is the heap h_c of m_c when the current function was called, or the initial heap h_{init} for the main frame. We write $\sigma \rightsquigarrow \sigma'$ for a transition of the concrete semantics in §1.

3.2 Concretization

The extended abstract domain refers to symbols and variables, so its concretization is defined relative to a heap h, an environment ρ, and a symbol valuation.

Definition 3.1 (Symbol Valuation). A symbol valuation $\eta : \Omega \to \mathbb{V}$ maps each symbol ω^{k} to the value of the k-th argument when the current function was called.

Since a symbol denotes an argument *value*, its meaning does not depend on the heap and never changes while the callee runs.

Definition 3.2 (Symbolic Type Denotation). The set of values $[\![\tau^\omega]\!]^\eta_h \subseteq \mathbb{V}$ described by a symbolic type τ^ω is defined as follows:

$$
\begin{aligned}
v \in [\![\tau]\!]^\eta_h &\iff (h, v) \in \gamma_\tau(\tau) \\
v \in [\![\omega]\!]^\eta_h &\iff v = \eta(\omega) \\
v \in [\![\tau^\omega.\mathtt{f}]\!]^\eta_h &\iff \exists a.\ a \in [\![\tau^\omega]\!]^\eta_h \wedge h(a)(\mathtt{f}) = v \\
a \in [\![\{\overline{\mathtt{f}_k : \tau^\omega_k}\}]\!]^\eta_h &\iff \forall k.\ \mathtt{f}_k \in \mathrm{dom}(h(a)) \wedge h(a)(\mathtt{f}_k) \in [\![\tau^\omega_k]\!]^\eta_h
\end{aligned}
$$

Definition 3.3 (Satisfaction of Type Propositions and Type Guards).

$$
\begin{aligned}
(h, \rho, \eta) &\models \top_\psi \quad \text{always} \qquad (h, \rho, \eta) \models \bot_\psi \text{ never} \\
(h, \rho, \eta) &\models x : \tau \iff x \in \mathrm{dom}(\rho) \wedge (h, \rho(x)) \in \gamma_\tau(\tau) \\
(h, \rho, \eta) &\models \omega <: \tau \iff (h, \eta(\omega)) \in \gamma_\tau(\tau) \\
(h, \rho, \eta) &\models \psi_1 \wedge \psi_2 \iff (h, \rho, \eta) \models \psi_1 \wedge (h, \rho, \eta) \models \psi_2 \\
(h, \rho, \eta, v) &\models G \iff \forall \tau^T \in \mathcal{T}.\ (h, v) \in \gamma_\tau(\tau^T) \Rightarrow (h, \rho, \eta) \models G(\tau^T)
\end{aligned}
$$

A type guard is a set of conditional promises: if the value belongs to a target type τ^T, the proposition $G(\tau^T)$ holds. Both the membership and the proposition are judged in the *same* heap h. This flow-sensitive reading is why the analysis weakens both type propositions and type guards when the heap changes.

Definition 3.4 (Abstract Values and Environments).

$$v \in \gamma^{\eta}_{h,\rho}(\tau^{\omega}[G]) \iff v \in [\![\tau^{\omega}]\!]^{\eta}_{h} \wedge (h, \rho, \eta, v) \models G$$
$$\rho \in \gamma^{\eta}_{h}(\widehat{\rho}) \iff \forall x \in \mathrm{dom}(\rho).\ x \in \mathrm{dom}(\widehat{\rho}) \wedge \rho(x) \in \gamma^{\eta}_{h,\rho}(\widehat{\rho}(x))$$

Definition 3.5 (Heap Evolution). A heap h evolves into h' within an effect δ, written $h \preceq_{\delta} h'$, if every address allocated in h is still allocated in h', and every field $\mathtt{f}$ of such an address whose value differs between h and h' satisfies $\mathtt{f} \in \mathrm{dom}(\delta)$ and $(h', h'(a)(\mathtt{f})) \in \gamma_{\tau}(\delta(\mathtt{f}))$.

Definition 3.6 (Frame Coverage). A typing result $\Gamma = (\widehat{\rho}, \psi, \delta)$ covers a memory $m = (h, \rho)$ with a symbol valuation η and an entry heap h_0, written $(\eta, h_0) \vdash m : \Gamma$, if $\rho \in \gamma^{\eta}_{h}(\widehat{\rho})$, $(h, \rho, \eta) \models \psi$, and $h_0 \preceq_{\delta} h$.

The last condition makes the effect δ of a typing result an over-approximation of all field writes since the entry of the current function. It is the invariant that justifies applying the callee effect to the caller when returning from a function.

Definition 3.7 (State Coverage). An analysis result $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ covers a concrete state $\sigma = (\kappa, l, m)$ with an initial heap h_{init}, written $\sigma \in \gamma(\widehat{\sigma})$, if

(1) m is well-formed (every address stored in the environment or the heap is allocated, and every function value belongs to the program);
(2) there exists a well-formed symbol valuation η such that $(\eta, h_0) \vdash m : \xi(l)$ where h_0 is the entry heap of the frame, and if $\kappa = (\kappa_c, l_c, m_c)$, then $\eta(\omega^{\mathsf{k}})$ is the value of the k-th argument of the call at l_c evaluated in m_c;
(3) if $\kappa = (\kappa_c, l_c, m_c)$, then $l_c \in \widehat{\kappa}(\Lambda)$ for the function Λ containing l, and the caller state (κ_c, l_c, m_c) is covered by $\widehat{\sigma}$ as well.

The recursion in the last condition covers every frame on the call stack, each with its own symbol valuation and entry heap.

3.3 Valid Analysis Results

Instead of the least fixpoint of the abstract semantics, we state soundness for any analysis result that satisfies the transfer of every instruction as a constraint. This formulation only requires the post-fixpoint property that the proof actually uses.

Definition 3.8 (Semantic Order). $\Gamma_1 \sqsubseteq \Gamma_2$ if $(\eta, h_0) \vdash m : \Gamma_1$ implies $(\eta, h_0) \vdash m : \Gamma_2$ for all η, h_0, and m.

Definition 3.9 (Valid Analysis Result). An analysis result $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ is valid for a program P, written $\mathsf{Valid}(P, \widehat{\sigma})$, if the following holds for every label l with $\Gamma = \xi(l)$ and its next label l':

$x = e$	$: \Gamma.\widehat{\mathsf{update}}_x(x, \widehat{[\![e]\!]}_e(\Gamma)) \sqsubseteq \xi(l')$
$x = \{\overline{\mathtt{f}_k : e_k}\}$	$: \Gamma.\widehat{\mathsf{update}}_x(x, \{\overline{\mathtt{f}_k : \widehat{[\![e_k]\!]}_e(\Gamma)}\}.\widehat{\mathsf{bind}}(\psi)) \sqsubseteq \xi(l')$
$e_1.\mathtt{f} = e_2$	$: \Gamma.\widehat{\mathsf{update}}_{\mathtt{f}}(_, \mathtt{f}, \widehat{[\![e_2]\!]}_e(\Gamma)) \sqsubseteq \xi(l')$
$\mathtt{if}\ (e)\ l_t\ \mathtt{else}\ l_f$	$: \Gamma.\widehat{\mathsf{refine}}(e, \#\mathtt{t}) \sqsubseteq \xi(l_t) \wedge \Gamma.\widehat{\mathsf{refine}}(e, \top \setminus \#\mathtt{t}) \sqsubseteq \xi(l_f)$
$\mathtt{assert}\ e$	$: \Gamma.\widehat{\mathsf{refine}}(e, \#\mathtt{t}) \sqsubseteq \xi(l')$
$\mathtt{call}\ x = e_0(\overline{e_k})$	$: \forall \Lambda = \lambda(\overline{x_k}).l_\Lambda \in \mathsf{MayCall}(\Gamma, e_0).\ l \in \widehat{\kappa}(\Lambda) \wedge (\{\overline{x_k \mapsto \omega^{\mathsf{k}}}\}, \psi', \bot_\delta) \sqsubseteq \xi(l_\Lambda)$
$\mathtt{return}\ e$	$: \forall l_c \in \widehat{\kappa}(\mathsf{func}(l)).\ \Gamma_c.\widehat{\mathsf{update}}_x(x_c, \widehat{[\![e]\!]}_e(\Gamma).\mathsf{instantiate}(\xi(l_c), \overline{e_{c,j}}, \delta).\widehat{\mathsf{bind}}(\Gamma_c.\psi)) \sqsubseteq \xi(\mathsf{next}(l_c))$

where $\psi' = \bigwedge_k \omega^{\mathsf{k}} <: \lceil \tau^{\omega}_k \rceil_{\psi}$ for $\widehat{[\![e_k]\!]}_e(\Gamma) = \tau^{\omega}_k[_]$, $\mathsf{MayCall}(\Gamma, e_0)$ is the set of functions that e_0 may evaluate to in a memory covered by Γ, δ is the effect of Γ, $\mathtt{call}\ x_c = _(\overline{e_{c,j}})$ is the instruction at l_c, and $\Gamma_c = \xi(l_c).\widehat{\mathsf{apply}}(\delta)$.

The return constraint reflects the revised instantiation in Figure 12: the argument descriptions and guards come from the caller state $\xi(l_c)$ at the call, the callee effect δ weakens the propagated propositions, and the result is bound with the caller proposition after applying δ.

Valid analysis results always exist: the analysis result that maps every label to $(\lambda x.\top, \top_\psi, \lambda \mathtt{f}.\top)$ and every function to all labels is valid for every program, and it covers every well-formed initial state of the main function (`topAnalysis_valid`, `topAnalysis_covers_main`).

3.4 Soundness Theorem

Theorem 3.10 (Soundness). *Let P be a well-formed program and $\widehat{\sigma}$ an analysis result with Valid$(P, \widehat{\sigma})$. If $\sigma_0 \in \gamma(\widehat{\sigma})$ with the initial heap of σ_0 and $\sigma_0 \rightsquigarrow^* \sigma$, then $\sigma \in \gamma(\widehat{\sigma})$.*

Corollary 3.11 (Type Soundness of Variables). *Under the assumptions of Theorem 3.10, if $\sigma = (_, l, (h, \rho))$ is reachable, $\rho(x) = v$, and $\xi(l) = (\widehat{\rho}, \psi, _)$ with $\widehat{\rho}(x) = \tau^\omega[_]$, then $(h, v) \in \gamma_\tau(\lceil \tau^\omega \rceil_\psi)$.*

The theorem is a *coverage* soundness: the analysis result over-approximates every reachable state. Well-formedness of a program only requires that labels and functions are consistent (e.g., every branch target belongs to the same function); it does not require the absence of type errors.

3.5 Proof Structure

Theorem 3.10 follows by induction on $\rightsquigarrow^*$ from the preservation of $\gamma(\widehat{\sigma})$ by a single transition (`step_preserves`), and Corollary 3.11 follows from the soundness of upper type bounds (`upperTy_sound`). The preservation is proved by case analysis on the instruction with the following groups of lemmas.

(1) **Transport.** If $h \preceq_\delta h'$, the weakening weaken_δ preserves the meaning of types, type propositions, type guards, symbolic types, and abstract environments from h to h'. Heap evolution is reflexive, holds for a single field write $[\mathtt{f} \mapsto \tau]$ when the written value has type τ, and composes with the effect composition $\delta \fatsemi \delta'$. This group justifies the field update and the effect application.

(2) **Type Guard Operations.** Guard lookup, update, refinement, and join are sound with respect to guard satisfaction. The query-form field lookup $G.\mathtt{f}$ is sound because $h(a)(\mathtt{f}) \in \gamma_\tau(\tau^T)$ implies $a \in \gamma_\tau(\{\mathtt{f} : \tau^T\})$ for open records, and the record type guard is sound because a record in τ^T has field values in $\tau^T.\mathtt{f}_j$ (`tyField_sound`).

(3) **Expressions.** If a memory is covered by Γ and e evaluates to v, then $v \in \gamma^\eta_{h,\rho}(\widehat{[\![e]\!]}_e(\Gamma))$. This relies on the soundness of the type proposition inference $\widehat{\mathsf{infer}}_\psi$ (both the symbolic and the expression derivations) and of $\widehat{\mathsf{bind}}$.

(4) **Updates and Refinement.** $\widehat{\mathsf{update}}_x$, $\widehat{\mathsf{update}}_\mathtt{f}$, and $\widehat{\mathsf{refine}}$ are sound: after a variable update, propositions about the variable are removed; after a field update, the typing result is weakened with the single-field effect; and refinement only adds propositions that hold in the taken branch.

(5) **Calls and Returns.** A call establishes the coverage of the callee entry with η mapping each symbol to the argument value and with the caller heap as the entry heap. For a return, the callee coverage gives $h_c \preceq_\delta h_r$ from the caller heap h_c at the call to the heap h_r at the return. The stability lemma shows that each argument description $\hat{\tau}^\omega_k$ still denotes the argument value in h_r, and the instantiation lemma combines it with transport: the argument guards weakened by δ hold in h_r, and each callee proposition $\omega^{\mathsf{k}} <: \tau$, which holds for the argument value, is translated by $\widehat{\mathsf{infer}}_\psi$ into a caller proposition that holds in h_c and, after weaken_δ, in h_r. Finally, applying δ to the caller typing result and binding its proposition yields the coverage of the return site.

3.6 Relation to the Implementation

The theorem covers the formalized analysis, and we do not formally verify that the implementation follows it. The implementation differs from the formalization in two ways.

Precision Refinements. The implementation uses more information than the formalization at several points, each justified by the same argument as the corresponding lemma: 1) it looks up field guards with $\{\mathtt{f} : \tau^T\} \sqcap \lceil \tau^\omega \rceil_\psi$, where τ^ω is the symbolic type of the base record; 2) it keeps guard entries for record target types whose fields are not in the effect; 3) it records effects per pair of a record type name and a field name using the partitioned abstract addresses in §5 of the paper.

Soundy Approximations. The implementation makes three approximations inherited from the original implementation: 1) for a callee whose declared return type does not overlap with target types, the caller uses the declared return type without applying the callee effect; 2) a field write on an object whose record type is unknown records no effect; 3) mutations other than writing a constant field of a local variable (e.g., computed field writes and list mutations through aliases) do not weaken type propositions. These approximations may miss type errors, and replacing them with the formalized operations is future work.

References

[1] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming languages (POPL).* doi:10.1145/512950.512973

[2] Patrick Cousot and Radhia Cousot. 1992. Abstract Interpretation Frameworks. *Journal of Logic and Computation (JLC)* 2, 4 (1992), 511–547. doi:10.1093/logcom/2.4.511

[3] Jihyeok Park, Seungmin An, Wonho Shin, Yusung Sim, and Sukyoung Ryu. 2021. JSTAR: JavaScript Specification Type Analyzer using Refinement. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE).* doi:10.1109/ASE51524.2021.9678781